PROJECTE DASC SERVER MVP

Nombre de los integrantes : Aiman Fadli, Daniel Galvez, Santino Colmena y Carlos Cano

Nombre del tutor/a : Jesús Vázquez, Susana Barbero

Data de entrega: 6 de marzo de 2026

ÍNDICE

Introducción **5**
Infraestructura de red - Cisco Packet Tracer **6**
1. Descripción general 6
2. Topología de red 6
2.1 Zonas de red (VLAN) 6
3. Inventario de hardware y direccionamiento 7
4. Configuración Lógica y Seguridad 7
5. Pruebas de Conectividad (Validación) 8
Documentación API para el MVP **9**
Creación de la API desde cero 9
6. Configuración de contenedores Docker 15
7. Conexion al socket Docker 16
8. Configuración de base de datos de logs 16
9. Funcion log_event() 17
10. Middleware de logging automático 17
11. Función docker_ok() 18
12. Listado de servicios Docker 18
13. Acción sobre servicios 19
14. Ejecutar comandos dentro de contenedor 19
16. Función is_ok() 19
17. Rutas principales del panel 20
18. Listado de servicios 20
19. Acción sobre servicios 20
20. Endpoint API de backups 21
21. Test de conexión de backups 21
22. Ejecución de backup desde formulario 21
23. Función docker_exec() 21
24. Función run_backup_in_worker() 23
25. Función is_ok() 23
26. Ruta principal / 24
28. Ruta /logs 24
29. Ruta /servicios 24
30. Acción sobre servicios 24
31. Endpoint API /api/backups/{tipo} 25
32. Endpoint de prueba /api/test-backups 26
33. Endpoint /backups/run 26
Proceso **27**
Documentación técnica de Servicios **29**
Documentación técnica de Docker **38**
1. Objetivo de la solución Docker **38**

2. Visión general de la arquitectura ... 39
3. Servicios definidos en docker-compose.yml ... 40
4. Redes y volúmenes ... 41
5. Estructura de carpetas del proyecto ... 42
6. Explicación detallada de docker-compose.yml ... 43
7. Carpeta web/: panel FastAPI y control del stack ... 44
7.1 Dockerfile de web ... 44
7.2 main.py: explicación funcional orientada a Docker ... 44
8. Carpeta backup/: contenedor trabajador de copias ... 46
8.1 Dockerfile del worker ... 46
8.2 backups_api.sh explicado paso a paso ... 46
9. Carpeta mysql/init/: inicialización de la base principal ... 47
9.1 01-seed.sql ... 47
9.2 02-acceso.sql ... 47
10. Carpeta logsdb/init/: auditoría centralizada ... 48
11. Carpeta cacti-db/init/ y carpeta cacti/: monitorización contenerizada ... 49
11.1 cacti-db/init/01-timezones-and-grants.sh ... 49
11.2 cacti/Dockerfile ... 49
11.3 debian.php.template ... 49
11.4 entrypoint.sh, explicado paso a paso ... 49
12. Comandos de trabajo: construcción, arranque y verificación ... 51
12.1 Preparación inicial ... 51
12.2 Comprobación del estado ... 51
12.3 Accesos funcionales ... 51
12.4 Rebuild selectivo ... 51
12.5 Logs y depuración ... 52
12.6 Parada y limpieza ... 52
13. Flujo de funcionamiento del stack, de punta a punta ... 53
14. Justificación técnica de las decisiones de diseño ... 54
15. Incidencias reales y consideraciones prácticas ... 54
16. Conclusión ... 55
Resumen fichero por fichero ... 56
Documentación técnica de Backups ... 57
1. Contexto y alcance del módulo de backups ... 57
2. Situación del módulo dentro del proyecto DASC Server Manager ... 57
3. Arquitectura real por máquinas ... 58
Flujo lógico del módulo de backups ... 59
4. Estructura de carpetas, ficheros y responsabilidades ... 60
4.1 Motivo de cada elemento ... 61
5. Decisiones técnicas del MVP ... 61
6. Implementación paso a paso ... 63
6.1 Preparación de MariaDB en la máquina 192.168.60.20 ... 63
6.2 Preparación de la máquina 192.168.60.30 ... 64
6.3 Preparación de la API del panel en 192.168.60.10 ... 65

6.4 Construcción de la interfaz de usuario ... 65
7. Explicación línea a línea del backend FastAPI ... 66
8. Explicación línea a línea del script Bash de backups ... 68
8.1 Encabezado y seguridad básica del script ... 69
8.2 Recepción de argumentos ... 69
8.3 Validación de mínimos obligatorios ... 69
8.4 Creación de carpeta y construcción del nombre seguro ... 69
8.5 Línea de extracción de datos ... 69
8.6 Compresión y retención ... 70
9. Flujo completo de ejecución desde la web hasta el fichero final ... 70
10. Pruebas de validación realizadas ... 72
11. Incidencias reales y resolución ... 73
12. Estado final del MVP y límites asumidos ... 74
13. Mejoras previstas para la versión final ... 76
14 Comandos representativos del montaje ... 76
Documentación de Logs ... 77
1. Objetivo del módulo de logs ... 77
2. Decisión técnica adoptada ... 77
3. Trabajo realizado con Cacti ... 77
4. Problemas reales y solución aplicada ... 77
5. Estado final de Cacti ... 78
6. Trabajo realizado con Grafana ... 78
7. Estructura resultante del módulo de logs ... 78
Evolución del proyecto y visión de la versión final ... 79
1. Sistema completo de restauración de backups ... 79
2. Programación automática de copias de seguridad ... 80
3. Implementación completa de backups incrementales y diferenciales ... 80
4. Sistema de autenticación y control de acceso ... 81
5. Sistema de alertas y notificaciones ... 81
6. Ampliación del sistema de monitorización ... 82
7. Mejora de la interfaz web ... 82
8. Instalación y despliegue automatizado ... 82
Gestión de Tareas ... 84

Introducción

En este proyecto hemos desarrollado DASC Server Manager, un MVP (Minimum Viable Product) cuyo objetivo es crear un sistema que permita gestionar servicios, realizar copias de seguridad y monitorizar servidores desde un panel web centralizado. Este trabajo lo hemos realizado en grupo, aplicando conocimientos de redes, sistemas, programación y virtualización para construir una solución funcional que simule un entorno real de administración de servidores.

Para empezar, diseñamos una infraestructura de red segmentada mediante VLANs, utilizando Cisco Packet Tracer, con el objetivo de organizar la red en diferentes zonas y mejorar la seguridad. De esta manera se separan los usuarios, los servidores, la zona de gestión y la DMZ, permitiendo controlar mejor el tráfico entre los distintos sistemas.

Después desarrollamos la API del proyecto utilizando FastAPI en Python, que actúa como núcleo del sistema. Esta API se encarga de servir el panel web, procesar las peticiones de los usuarios y ejecutar acciones administrativas, como controlar servicios o lanzar copias de seguridad en los servidores.

También implementamos una arquitectura basada en Docker, donde cada servicio del sistema se ejecuta en un contenedor independiente. Esto nos permitió organizar mejor el proyecto, facilitar el despliegue del sistema y simular una infraestructura más realista. Dentro de esta arquitectura se incluyen componentes como la base de datos, el sistema de backups, el panel web y la monitorización del sistema.

Además, desarrollamos un módulo de backups automatizados, capaz de realizar copias de seguridad de bases de datos, y un sistema de registro de eventos (logs) que permite almacenar información sobre las acciones realizadas dentro del panel.

En este documento explicamos todo el proceso de desarrollo del proyecto: el diseño de la red, la arquitectura del sistema, la implementación de la API, la configuración de Docker y el funcionamiento de cada módulo. Nuestro objetivo con este trabajo ha sido crear una solución funcional que demuestre

cómo se pueden integrar diferentes tecnologías para gestionar servidores de forma centralizada y segura.

Infraestructura de red - Cisco Packet Tracer

1. Descripción general

La infraestructura de red ha sido diseñada para soportar un sistema de gestion de backups y servicios automatizados. Hemos optado por un modelo de red segmentada mediante VLAN's y un firewall perimetral, garantizando que los datos críticos (como mysql) y las herramientas de gestión de backups no estén expuestos directamente a redes no autorizadas

2. Topología de red

La red se organiza en 4 zonas principales de seguridad, conectadas a través de un switch multicapa que actúa como el motor de enrutamiento interno

2.1 Zonas de red (VLAN)

VLAN	Nombre	Subred	Propósito
10	DMZ	192.168.10.0/24	Aloja la API (FastAPI) y es una zona accesible para los usuarios.
20	Servers	192.168.20.0/24	Zona aislada para la base de datos MySQL.
30	MGMT/SSH	192.168.30.0/24	Segmento donde reside el servidor de backups y ejecución de scripts SSH.
40	Users	192.168.40.0/24	Estaciones de trabajo de los empleados y los administradores.

3. Inventario de hardware y direccionamiento

Dispositivo	Modelo	Interfaz
Firewall ASA	ASA 5505	Outside (VLAN2)
Firewall ASA	ASA 5505	Inside (VLAN1)
Switch Core	3560	SVI 10,20,30,40
Servidor API	Generico	Eth0
Servidor MySQL	Generico	Eth0
Servidor Backup	Generico	Eth0
PC	FastEth	FastEth0

4. Configuración Lógica y Seguridad

4.1. Enrutamiento Inter-VLAN (Switch Core)

Se ha configurado el comando ip routing en el Switch Multicapa 3560 para permitir la comunicación controlada entre segmentos. Cada VLAN dispone de una interfaz virtual (SVI) que sirve de puerta de enlace para los hosts finales.

4.2. Flujo de Tráfico

El diseño de red soporta estrictamente el flujo operativo del software desarrollado:

1. **Acceso Web:** Los usuarios (VLAN 40) acceden vía HTTP/HTTPS al Panel (VLAN 10).
2. **Gestión SSH:** La API (VLAN 10) solicita acciones al Servidor de Backups (VLAN 30) mediante el protocolo SSH (Puerto 22).
3. **Extracción de Datos:** El Servidor de Backups (VLAN 30) realiza un mysqldump conectándose al Servidor MySQL (VLAN 20) por el Puerto 3306.

4.3. Seguridad Perimetral (ASA)

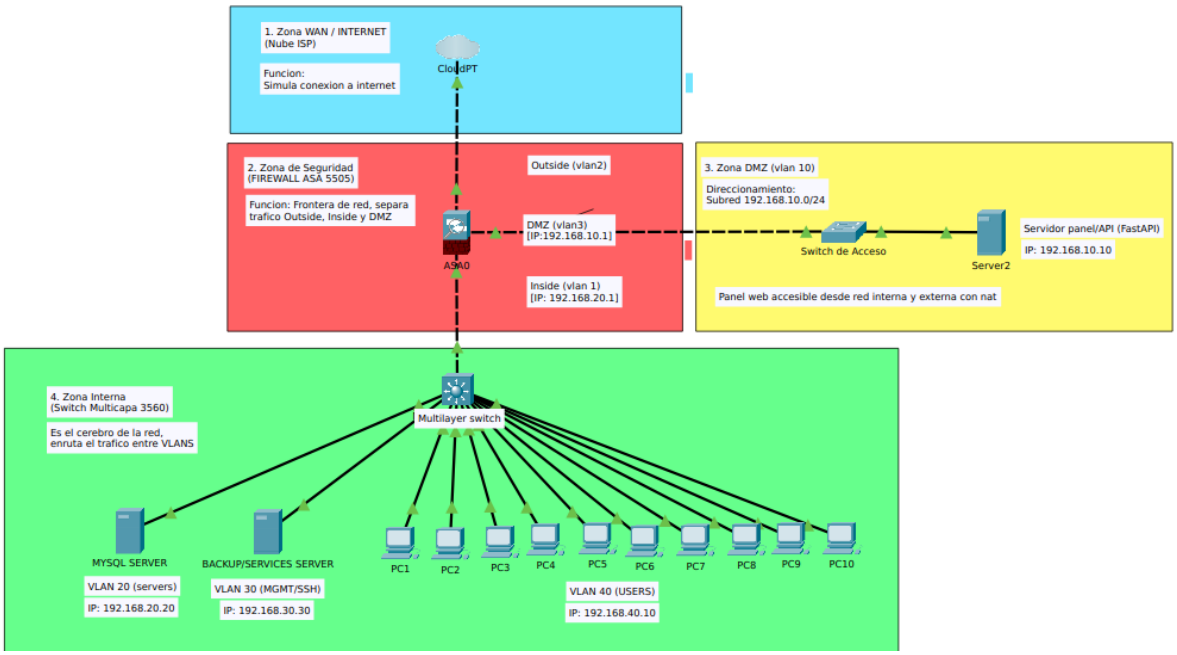
El Firewall ASA 5505 protege la red interna de la zona WAN (Internet). Se han definido niveles de seguridad (security-level) para asegurar que el tráfico solo fluya de zonas más seguras a menos seguras, a menos que se definan reglas explícitas.

- Outside (0): Internet.
- DMZ (50): Panel Web.
- Inside (100): Red de Servidores y Usuarios.

5. Pruebas de Conectividad (Validación)

Para asegurar el correcto funcionamiento, se realizaron las siguientes pruebas de conectividad (ICMP):

- PC Admin → Servidor API: Exitosa. El administrador puede gestionar el panel.
- Servidor API → Servidor Backup: Exitosa. Permite el envío de órdenes de ejecución.
- Servidor Backup → MySQL: Exitosa. Permite la extracción remota de bases de datos.
- Internet → MySQL: Bloqueada por Firewall (Seguridad por diseño).



Documentación API para el MVP

Creación de la API desde cero

En este apartado se documenta todo el proceso de creación de API del proyecto, desde la instalación inicial del entorno hasta la estructura final del sistema.

La API se ha desarrollado usando Fast API, que permite crear servicios web rápidos en Python y además servir HTML mediante Jinja2.

La api cumple dos funciones principales:

- Servir el panel web del sistema
- Ejecutar acciones remotas en servidores mediante SSH

Instalacion del entorno base

El primer paso fue preparar el sistema ubuntu con las herramientas necesarias para desarrollar la API

1.1 Actualizar repositorios del sistema
Sudo apt update

1.2 Instalar dependencias del sistema
Sudo apt install -y python3 python3-venv python3-pip openssh-client

Se instalaron los siguientes paquetes:

Paquete	Funcion
Python 3	Lenguaje de programación usado para la Api
python3-venv	Permite crear entorno virtuales de Python
python3-pip	Gestor de paquetes de Python
openssh-client	Permite ejecutar comandos SSH desde la API

El cliente SSH es necesario porque el API ejecuta scripts remotos en otros servidores.

Crear estructura del proyecto

Una vez preparado el sistema se creó la carpeta donde se almacenará el proyecto.
mkdir -p ~/Proyecto/Api/api-automatizacion

Después se asignó la propiedad al usuario actual, con esto evitamos problemas de permisos al editar los archivos.
chown $USER:$USER ~/Proyecto/Api/api-automatizacion

Accedemos al directorio
cd ~/Proyecto/Api/api-automatizacion
Creación del entorno virtual Python

Para aislar la dependencias del proyecto se creo un entorno virtual
python3 -m venv .venv

Esto genera la carpeta
.venv/

Activar el entorno virtual

- Para usar el entorno visual se ejecuta → source .venv/bin/activate
- Cuando está activo el prompt cambia a algo como → (venv) user@server:$

3.2 Porque usamos un entorno virtual
El entorno virtual sirve para
- Instalar librerias solo para este proyecto
- Evitar conflictos con otros proyectos
- No modificar el Python del sistema
- Permitirá reproducir el proyecto fácilmente

Si no usaramos venv tendremos más problemas.

3.3 Contenido de la carpeta .venv
Dentro del entorno de virtual aparecen varias carpetas:

```
.venv/
    bin/
    include/
    lib/
    pyvenv.cfg
```

Explicacion

Carpeta	Funcion
bin	Ejecutables dentro del entorno virtual
include	Cabeceras necesarias para compilar librerías

lib	Librerias Python instaladas
pyvenv.conf	Configuración del entorno virtual

Instalación de dependencias de Python

Con el entorno virtual activo se instalaron las librerías necesarias para el proyecto
pip install fastapi uvicorn jinja2 python-multipart itsdangerous

Funcion de cada paquete

Paquete	***Funcion***
Fast Api	Framework web
Uvicorn	Servidor ASGI
Jinja2	Motor de plantillas
Python-multipart	Leer form
Itsdangerous	Necesario para login

Creación de la estructura de carpetas del proyecto

Para organizar correctamente el código se creó la siguiente estructura.
mkdir -p templates
mkdir -p static/css
mkdir -p static/img

Esto genera:

```
api-automatizacion/
    templates
    static
        css
        img
```

Carpeta Templates
La carpeta templates contiene los archivos HTML del panel web

Estos archivos contienen el frontend del panel

¿Por qué usamos templates?
Fast Api utiliza Jinja2 pare renderizar HTML dinámico, esto permite insertar variables dentro del HTML

Relación entre la API y la interfaz web

La API no solo actúa como backend para procesar peticiones, sino que también es la encargada de servir la interfaz web del sistema. El panel de control del proyecto se ha construido utilizando HTML y CSS, organizados en la carpeta templates para las plantillas y en static para los archivos de estilo e imágenes.

Los archivos HTML funcionan como plantillas Jinja2, lo que permite que el backend envíe información dinámica desde Python hacia la interfaz web. De esta forma el panel puede mostrar datos como el estado de los servicios, los resultados de las copias de seguridad o los registros del sistema.

La estructura utilizada es la siguiente:

```
templates/
  index.html
  servicios.html
  backups.html
  logs.html

static/
  css/
    estilo.css
    servicios.css
    backups.css
  img/
```

Carpeta Static
La carpeta static contiene archivos estaticos

Estructura

```
static/
    css
        estilo.css
        backups.css
        servicios.css
    img
```

Aqui se almacena
- Hojas de estilo
- Imagenes
- Iconos

Por que usa static
Fast Api necesita montar una carpeta para servir archivos estáticos

Archivo main.py

El archivo main.py es el componente principal de la aplicaicon, el corazon de nuestra API
Este archivo se encarga de
- Crear el servidor FAST API
- Gestionar servicios Docker
- Ejecutar backups dentro de contenedores
- Servir el frontend HTML
- Gestionar logs centralizados en MySQL
- Controlar peticiones mediante HTTP
- Ejecutar scripts remotos mediante SSH

En resumen main.py conecta la interfaz web con la infraestructura del sistema

1. Importación de librerías

from fastapi import FastAPI, Request, Form
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import RedirectResponse, JSONResponse
from urllib.parse import quote
import os
import pymysql
from datetime import datetime

import docker
from docker.errors import NotFound, APIError

Este bloque se dedica a importar las librerías necesarias para que la API funcione

Explicación de cada una de ellas

Libreria	Funcion
FastAPI	Framework web principal
Request	Permite acceder a datos de la peticion
Form	Permite leer formularios HTML
Jinja2Templates	Motor de plantillas HTML
StaticFiles	Permite servir CSS e imagenes
CORSMiddleware	Permitir peticiones desde otro dominios
RedirectResponse	Redirecciones despues de formularios
JSONResponse	Respuestas en formato JSON
urllib.parse-quote	Codifica mensaje para URL
Os	Leer variables de entorno
Pymysql	Conectar con base de datos MySQL
Datetime	Genera timestamps
Docker	Controlar contenedores
Docker.errors	Manejo de errores de Docker

2. Creacion del servidor Fast API

app = FastAPI() → Esto crea la instancia principal del sevidor web, todas las rutas @app.get, @app.post) se resgitraran aqui.

3. Configuracion del motor de plantillas HTML

templates = Jinja2Templates(directory="templates") → *Esto indica a FastAPI que los archivos HTML están en la carpeta templates/.*

Se usa jinja2 para renderizar HTML dinâmico.

Ejemplo → templates.TemplatesResponde("index.html", {"request": request}) → Permite pasar variables desde Python al HTML

4. Configuracion de archivos estaticos

app.mount("/static", StaticFiles(directory="static"), name="static") → Esto sirve archivos CSS, JS e imagenes

5. Configuración CORS

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Cors permite que otras aplicaciones accedan a la API, en nuestro MVP se deja abierto → allow_origins=[""]. Esto evita problemas si en el futuro el frontend esta en otro servidor aunque normalmente se restringe.*

6. Configuración de contenedores Docker

```
MANAGED_CONTAINERS = os.getenv(
    "MANAGED_CONTAINERS",
    "dasc-web,mysql-db,backup-worker,cacti"
).split(",")
```

Define qué contenedores puede controlar la APi, en nuestro caso tenemos 4 contenedores sin contar el de la API:

- Dasc-web
- Mysql-db
- Backup-worker
- Cacti

Gracias a esto podemos evitar que nuestro panel pueda manipular contenedores no autorizados.

6.1 Contenedores de backups

```
BACKUP_CONTAINER = os.getenv("BACKUP_CONTAINER", "backup-worker")
BACKUP_SCRIPT = os.getenv("BACKUP_SCRIPT", "/scripts/backups_api.sh")
```

Indica que contenedor ejecutar backups y que scripts ejecuta el backup

6.2 URL CACTI

```
CACTI_URL = os.getenv("CACTI_URL", "http://127.0.0.1:8081/")
```

Define la URL del sistema de monitorización cacti, que despues de pasa al HTML de logs.

7. Conexion al socket Docker

```
docker_client = docker.DockerClient(base_url="unix://var/run/docker.sock")
```

Esto conecta la API con el daemon Docker del sistema.

El socket:

```
/var/run/docker.sock
```

Esto permite:

- *listar contenedores*
- *arrancar contenedores*
- *parar contenedores*
- *ejecutar comandos dentro de contenedores*

8. Configuración de base de datos de logs

```
LOGS_DB_HOST = os.getenv("LOGS_DB_HOST", "logs-db")
LOGS_DB_NAME = os.getenv("LOGS_DB_NAME", "dasc_logs")
LOGS_DB_USER = os.getenv("LOGS_DB_USER", "dasc")
LOGS_DB_PASS = os.getenv("LOGS_DB_PASS", "dascpass")
LOGS_ORIGIN  = os.getenv("LOGS_ORIGIN", "dasc-web")
```

Define la base de datos donde se guardan logs de eventos del sistema.

Esto permite registrar:

- ***accesos al panel***
- ***ejecución de servicios***
- ***ejecución de backups***
- ***errores***

9. Funcion log_event()

```
def log_event(...)
```

Esta función guarda eventos en la base de datos MySQL.

Ejemplo de evento registrado

```
tipo: servicio
usuario: anon
ip_origen: 192.168.1.20
recurso: POST /servicios/accion
resultado: OK
```

Consulta SQL que ejecuta:

```
INSERT INTO eventos(...)
VALUES(...)
```

*Si ocurre un error en el log **se ignora**:*

```
except Exception:
    pass
```

Esto evita que un fallo de logs rompa la API.

10. Middleware de logging automático

```
@app.middleware("http")
async def log_requests(request: Request, call_next):
```

Este middleware intercepta todas las peticiones HTTP.

Flujo:

1. Llega una petición

2. Se procesa normalmente

3. Se registra en la base de datos

Qué registra

Información	*Ejemplo*
IP cliente	*192.168.1.20*
Ruta	*/servicios*
Método	*GET*
Resultado	*OK / ERROR*

Ignora archivos static

```
if path.startswith("/static"):
    return await call_next(request)
```

No registra CSS ni imágenes

11. Función docker_ok()

```
#Comprueba si el socket Docker está accesible.
def docker_ok()

#Ejecuta
docker_client.ping()

#Devuelve
(True, "OK")

#Si da error devuelve
(False, error)
```

Esto evita que la API falle si Docker no está activo.

12. Listado de servicios Docker

#Obtiene el estado de los contenedores definidos.
def docker_ps_services()

#Para cada contenedor
docker_client.containers.get(name)

#Obtiene estado
running → Activo

stopped → Inactivo

#Devuelve lista
[
{"nombre":"mysql-db","estado":"Activo"},
{"nombre":"cacti","estado":"Inactivo"}
]

13. Acción sobre servicios

def docker_service_action(service, action)

Permite ejecutar acciones

Start/stop/restart

Primero valida:

- *que el servicio está permitido*
- *que la acción es válida*

Protección especial

if service == "dasc-web"

No permite parar el contenedor del panel .Esto evita que la propia web se ***apague a sí misma***

14. Ejecutar comandos dentro de contenedor

def docker_exec(container_name, cmd)

Ejecuta comandos dentro de contenedor

Ejemplo:

```
docker exec backup-worker /scripts/backups_api.sh
```

Esto se usa para lanzar scripts de backup.

15. Ejecutar backup en worker

```
def run_backup_in_worker(args)
```

Llama al script de backup dentro del contenedor.

Ejecuta con los argumentos enviados:

```
/scripts/backups_api.sh
```

16. Función is_ok()

```
def is_ok(output)
```

Determina si el resultado del backup fue correcto

Busca palabras como:

```
OK
backup creado
created
```

17. Rutas principales del panel

```
#Home
@app.get("/")
```

Carga el panel principal index.htm

Backups

```
@app.get("/backups")
```

Carga backups.html y muestra el estado del último backup

Logs

```
@app.get("/logs")
```

Carga logs.html y pasa la URL de Cacti

18. Listado de servicios

```
@app.get("/servicios")
```

Obtiene contenedores con docker_ps_services() y los envía al HTML

19. Acción sobre servicios

```
@app.post("/servicios/accion")
```

Recibe formulario HTML

Ejemplo:

```
service=mysql-db
action=restart
```

Ejecuta acción y redirige

20. Endpoint API de backups

```
@app.post("/api/backups/{tipo}")
```

Permite lanzar backups desde API tipo full/incremental/differential

21. Test de conexión de backups

```
@app.post("/api/test-backups")
```

Comprueba que:

- *Docker funciona*
- *el contenedor worker responde*

Ejecuta hostname && date

22. Ejecución de backup desde formulario

```
@app.post("/backups/run")
```

Este endpoint recibe los datos del formulario

Ejemplo

```
db
dest
name
compress
retention
```

Luego ejecuta backups_api.sh y redirige al panel

23. Función docker_exec()

```
def docker_exec(container_name: str, cmd: list[str]) -> tuple[int, str]:
```

Esta función permite ejecutar comandos dentro de un contenedor Docker desde la API

Es equivalente a ejecutar en terminal docker exec <contenedor> <comando>Parámetros

Parámetro	Descripción
container_name	nombre del contenedor
cmd	comando a ejecutar dentro del contenedor

Ejemplo de llamada

```
docker_exec("backup-worker", ["/scripts/backups_api.sh"])
```

Funcionamiento

```
1. Comprueba que Docker está accesible usando
docker_ok()

2. Obtiene el contenedor
docker_client.containers.get(container_name)

3. Ejecuta el comando dentro del contenedor
res = c.exec_run(cmd)
```

4. Devuelve

Valor	*Descripción*
exit_code	***código de salida del comando***
output	***texto generado por el comando***

Ejemplo de retorno

```
(0, "Backup creado correctamente")
```

Si hay error

```
(1, "ERROR: contenedor no encontrado")
```

24. Función run_backup_in_worker()

```
def run_backup_in_worker(args: list[str]) -> str:
```

Esta función ejecuta el script de backups dentro del contenedor backup-worker.

El contenedor worker se usa para separar la ejecución de backups del servidor web, lo que mejora la seguridad y estabilidad.

Construcción del comando

```
cmd = [BACKUP_SCRIPT] + args
```

Ejemplo de comando final → /scripts/backups_api.sh full dascdb /backups backup.sql none 7

Ejecución

La función utiliza internamente → ***docker_exec(BACKUP_CONTAINER, cmd)****, Esto ejecuta el script dentro del contenedor*

Resultado

Si el comando falla → ERROR (1): fallo en backup

Si funciona → Backup creado correctamente

25. Función is_ok()

```
def is_ok(output: str) -> bool:
```

Esta función analiza el texto devuelto por el script de backup para determinar si el proceso fue exitoso. Esto es necesario porque el script devuelve texto y no siempre un código estructurado.

Palabras clave que detecta

La función busca palabras como:

- *OK*
- *backup creado*
- *created*

Si encuentra alguna devuelve → True

Si no detecta ninguna → False

Esto permite que el panel web muestre si el backup fue exitoso o no.

26. Ruta principal /

```
#Esta ruta carga la página principal del panel web
@app.get("/")
return templates.TemplateResponse("index.html", {"request": request})
```

Funcionamiento

1. El usuario accede a → http://servidor/

***2. FastAPI** renderiza el HTML → templates/index.html*

3. Se envía al navegador

28. Ruta /logs

```
@app.get("/logs")
```

Carga la página de monitorización →templates/logs.html

También envía la variable → cacti_url , Esto permite incrustar el sistema de monitorización Cacti dentro del panel web

29. Ruta /servicios

```
@app.get("/servicios")
```

Esta ruta muestra el estado de los contenedores Docker.

Primero obtiene los contenedores:

```
servicios = docker_ps_services()
```

Ejemplo de datos enviados al HTML

```
[
 {nombre: mysql-db, estado: Activo},
 {nombre: cacti, estado: Inactivo}
]
```

Después se renderiza → ***templates/servicios.html,*** *esto permite ver desde el panel si los servicios están activos.*

30. Acción sobre servicios

```
@app.post("/servicios/accion")
```

Esta ruta recibe acciones desde un formulario HTML.

Ejemplo de formulario

service=mysql-db

action=restart

Proceso

1. Recibe los datos

```
service: str = Form(...)
action: str = Form(...)
```

2. Ejecuta la acción

```
docker_service_action(service, action)
```

3. Redirige al panel

```
/servicios?ok=1&msg=OK
```

Esto permite controlar los contenedores desde la interfaz web

31. Endpoint API /api/backups/{tipo}

```
@app.post("/api/backups/{tipo}")
```

Este endpoint permite ejecutar backups desde la API.

Tipos de backup permitidos:

- *full*
- *incremental*
- *differential*

Ejemplo de petición → POST /api/backups/full

La API ejecuta el script de backup con parámetros predefinidos.

Respuesta JSON:

```
{
 "ok": true,
 "tipo": "full",
 "resultado": "Backup creado",
 "timestamp": "2025-03-12T10:00:00"
}
```

Esto permite automatizar backups desde scripts o herramientas externas

32. Endpoint de prueba /api/test-backups

```
@app.post("/api/test-backups")
```

Este endpoint permite comprobar que el sistema de backups funciona correctamente.

Verificaciones que realiza

1. Comprobar conexión con Docker.

2. Comprueba que el contenedor backup-worker está accesible.

3. Ejecuta dentro del contenedor → hostname && date

Si responde correctamente significa que el sistema de ejecución remota funciona.

Ejemplo de respuesta

```
{
 "ok": true,
 "resultado": "backup-worker\nTue Mar 12 10:01:22 UTC 2025"
}
```

33. Endpoint /backups/run

```
@app.post("/backups/run")
```

Este endpoint recibe los datos del formulario HTML de creación de backups.

Datos recibidos:

Campo	Descripción
type	tipo de backup
db	base de datos
dest	destino
name	nombre archivo
compress	tipo compresión
retention	días de retención

Ejemplo de formulario

```
type=full
db=dascdb
name=backup.sql
compress=gzip
retention=7
```

Proceso

1. Valida tipo de backup.

2. Ejecuta → backups_api.sh

3. Redirige al panel → /backups?ok=1&msg=backup+creado

Esto permite lanzar backups desde la interfaz web

Arquitectura final del sistema

El flujo final del sistema es el siguiente:

```
Usuario


Navegador Web


FastAPI (main.py)
        Templates HTML (panel web)
        Archivos estáticos CSS/IMG
        Docker API
            mysql-db
            backup-worker
            cacti
         Base de datos MySQL (logs)
         Scripts de backup
```

La API actúa como punto central de control del sistema, conectando:

- *la interfaz web*
- *los contenedores Docker*
- *el sistema de backups*
- *el sistema de logs*
- *el sistema de monitorización*

Documentación técnica de Servicios

Script de servicios

Al principio del proyecto, ya que nuestra idea principal era trabajar usando servidores en máquinas y no mediante docker como hemos acabado haciendo, teníamos todas los scrips separados en su máquina correspondiente. Que desde la API se conectaba de manera SSH y seguimos estos pasos:

Paso 1. Crear la estructura de trabajo.

```
mkdir -p ~/dasc/templates
mkdir -p ~/dasc/static/css
mkdir -p ~/dasc/static/img
```

*Se separan las **plantillas** y los **archivos estáticos** porque FastAPI funciona con esa estructura:*

- *Jinja2Templates(directory="templates") para el HTML.*
- *StaticFiles(directory="static") para CSS e imágenes.*

Mantener esta separación evita mezclar el código del servidor con el diseño de la interfaz.

Paso 2. Copiar el backend principal y los ficheros de la interfaz.

```
cp main.py ~/dasc/main.py cp servicios.html ~/dasc/templates/servicios.html
 cp index.html ~/dasc/templates/index.html
cp logs.html ~/dasc/templates/logs.html
cp backups.html ~/dasc/templates/backups.html
cp estilo.css ~/dasc/static/css/estilo.css
cp servicios.css ~/dasc/static/css/servicios.css
```

El módulo de servicios no está separado del todo. La página de servicios usa el mismo estilo que el panel principal y otras vistas de la aplicación.

*Por eso se reutiliza **estilo.css** como hoja de estilos general, y se añade **servicios.css** solo para las reglas específicas de la tabla de servicios.*

Paso 3. Configurar variables de entorno.

```
export SSH_USER=dasc
export SERVICIOS_HOST=192.168.60.30
export BACKUPS_HOST=192.168.60.30
export CACTI_URL=/cacti/
```

Estas variables separan el código de los datos del laboratorio.
*Aunque **main.py** ya tiene valores por defecto, ponerlas de forma explícita permite mover el panel a otro ordenador o cambiar la IP del servidor sin tener que modificar el código.*

Paso 4. Arrancar la aplicación

```
cd ~/dasc uvicorn main:app
--host 0.0.0.0 --port 8000
```

Con este comando el panel se puede abrir desde el navegador. La ruta importante para este módulo es /servicios.

Para un despliegue más estable, se recomienda crear un servicio systemd para uvicorn, pero eso ya forma parte de la documentación general del proyecto y no del módulo específico de Servicios.

Preparación del servidor remoto de servicios

Paso 1. Confirmar que la máquina usa systemd y expone SSH.

```
sudo systemctl --version
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
hostname
ip a
```

*El módulo depende de **systemctl**, así que el servidor remoto debe ser una máquina **Ubuntu con systemd**.*

*También necesita tener **SSH activo**, porque la API no ejecuta el script en la máquina local: lo ejecuta de forma remota usando el usuario indicado en **SSH_USER**.*

Paso 2. Instalar el script de servicios

```
sudo cp servicios_api.sh /usr/local/bin/servicios_api.sh
sudo chmod 755 /usr/local/bin/servicios_api.sh
sudo chown root:root /usr/local/bin/servicios_api.sh
```

*La ruta **/usr/local/bin** es adecuada para guardar scripts administrativos propios.*

*Se dan permisos de **lectura y ejecución**, pero se evita que cualquier usuario pueda modificar el archivo. Así se reduce el riesgo de que alguien cambie por error el script que controla los servicios del sistema.*

Paso 3. Probar el listado manualmente antes de integrarlo con FastAPI.

```
/usr/local/bin/servicios_api.sh list | head
/usr/local/bin/servicios_api.sh start ssh.service
/usr/local/bin/servicios_api.sh restart cron.service
```

La comprobación manual es importante porque separa dos partes:

Primero se comprueba que el script Bash funciona bien. Después se comprueba su integración con la API.

Si el problema ya aparece en la consola, no tiene sentido abrir el navegador ni revisar el frontend todavía.

Paso 4. Configurar permisos sudo para acciones no interactivas

```
sudo visudo -f /etc/sudoers.d/dasc-servicios

dasc ALL=(ALL) NOPASSWD: /usr/bin/systemctl start *, /usr/bin/systemctl stop *,
/usr/bin/systemctl restart *
```

Este paso es crítico en un entorno real. El script ejecuta comandos como sudo systemctl start|stop|restart.

Si el usuario dasc necesita contraseña y no hay una terminal interactiva , la orden fallará cuando FastAPI la ejecute por SSH.

Por eso, aunque la regla exacta no aparezca en los archivos entregados, es una condición necesaria para que el módulo funcione desde el navegador.

Paso 5. Probar la conexión desde el servidor del panel.

```
ssh dasc@192.168.60.30 /usr/local/bin/servicios_api.sh list | head
ssh dasc@192.168.60.30 /usr/local/bin/servicios_api.sh restart ssh.service
```

Solo cuando esta prueba remota funciona tiene sentido abrir /servicios en el navegador.

Si falla aquí, el backend web también fallará, porque usa exactamente el mismo mecanismo por debajo.

Análisis detallado del script servicios_api.sh

El script Bash es la parte central del módulo porque es el que se comunica directamente con systemctl.

Su función es simple:

- *Listar servicios cuando recibe la acción list.*
- *Ejecutar start, stop o restart cuando recibe una acción de control.*

Esta simplicidad es una ventaja en un MVP, porque facilita la depuración y evita añadir lógica innecesaria.

```
#!/bin/bash
```

```
ACCION="$1"
SERVICIO="$2"

if [[ "$ACCION" == "list" ]]; then
    systemctl list-units --type=service --all --no-pager --no-legend | awk '{print $1}' | while
read service; do
        estado=$(systemctl is-active "$service")
        echo "$service|$estado"
    done
    exit 0
fi

if [[ "$ACCION" == "start" || "$ACCION" == "stop" || "$ACCION" == "restart" ]]; then
    sudo systemctl $ACCION "$SERVICIO"
    estado=$(systemctl is-active "$SERVICIO")
    echo "$SERVICIO|$estado"
    exit 0
fi

echo "Acción no válida"
exit 1
```

1. Explicación línea a línea del script Bash

Línea o bloque	***Explicación funcional***
Línea 1	*Shebang estándar. Indica que el script debe ejecutarse con Bash y no con sh u otro intérprete.*
Líneas 3-4	*Capturan los parámetros de entrada. El primer argumento se interpreta como acción y el segundo como nombre de servicio.*
Línea 6	*Primera condición. Si la acción recibida es list, el script entra en modo consulta y no en modo modificación.*
Línea 7	*systemctl list-units --type=service --all enumera todas las unidades de tipo servicio, incluso las inactivas. --no-pager evita paginación y --no-legend elimina la cabecera para facilitar el parseo.*
Línea 7, parte awk	*awk '{print $1}' se queda solo con el nombre de la unidad, descartando el resto de columnas que no son necesarias.*
Línea 7, while read service	*Recorre cada nombre de servicio uno por uno para poder consultar su estado de forma individual.*
Línea 8	*systemctl is-active devuelve active, inactive, failed, activating, etc. Se guarda en la variable estado.*
Línea 9	*La salida se normaliza como `nombre*
Línea 11	*exit 0 finaliza correctamente el modo listado, evitando que el script siga evaluando bloques que no corresponden.*
Línea 14	*Segunda condición. Solo se aceptan tres acciones operativas: start, stop y restart. Al validar aquí, se reduce la superficie de comandos permitidos.*

Línea 15	*La acción se aplica al servicio solicitado mediante sudo systemctl. sudo es necesario porque muchos servicios requieren privilegios elevados.*
Línea 16	*Después de la operación se consulta de nuevo el estado real, devolviendo un resultado actualizado y no una suposición.*
Línea 17	*El script responde con `servicio*
Línea 18	*exit 0 marca la finalización correcta del bloque operativo.*
Líneas 21-22	*Si no se ha cumplido ninguna condición anterior, la acción es inválida y el script termina con error. Esto protege frente a llamadas mal formadas.*

2. Por qué se eligió este formato y no otro

Se eligió una interfaz de comandos simple en lugar de JSON, XML o argumentos complejos porque el módulo no busca ser una API independiente, sino una herramienta de shell fácil de ejecutar por SSH.

Cuanto más simple sea el script, más fácil es revisarlo, explicarlo y mantenerlo. Además, al devolver la cadena nombre|estado, se consigue un buen equilibrio entre simplicidad y capacidad de integración.

3. Comportamiento real del script

Llamada	***Respuesta esperada***	***Uso real***
servicios_api.sh list	***Muchas líneas con `servicio***	***estado`***
servicios_api.sh start	***`nginx.service***	***active`***
servicios_api.sh stop	***`apache2.service***	***inactive`***

servicios_api.sh restart	***`cron.service***	***active`***
servicios_api.sh borrar	***Acción no válida***	***Protección frente a órdenes no permitidas***

Integración del script dentro de main.py (FastAPI)

FastAPI funciona como una capa intermedia entre el navegador y la máquina que gestiona los servicios.

No utiliza JavaScript para ejecutar acciones. En su lugar:

1. ***Recibe peticiones HTTP desde el navegador.***
2. ***Llama al script Bash por SSH.***
3. ***Renderiza la respuesta usando una plantilla Jinja2.***

Esta separación asegura que el frontend no tenga acceso directo al sistema operativo.

```
USUARIO = os.getenv("SSH_USER", "dasc")
SERVIDOR_SERVICIOS = os.getenv("SERVICIOS_HOST", "192.168.60.30")
SCRIPT_SERVICIOS = "/usr/local/bin/servicios_api.sh"
def ssh_run(host: str, script: str, args: list[str]) -> str:
 cmd = ["ssh", f"{USUARIO}@{host}", script] + args
 res = subprocess.run(cmd, capture_output=True, text=True)
 out = (res.stdout or "").strip()
 err = (res.stderr or "").strip()
 if res.returncode != 0:
 return f"ERROR ({res.returncode}): {err or out}"
 return out
@app.get("/servicios")
def ver_servicios(request: Request):
salida = ssh_run(SERVIDOR_SERVICIOS, SCRIPT_SERVICIOS, ["list"])
lista_servicios = []
for linea in salida.split("
"):
    if "|" in linea:
        nombre, estado = linea.split("|", 1)
        lista_servicios.append({"nombre": nombre.strip(), "estado": estado.strip()})
return templates.TemplateResponse("servicios.html", {"request": request, "servicios":
lista_servicios})
```

```
@app.post("/servicios/accion")
def accion_servicio(service: str = Form(...), action: str = Form(...)):
    ssh_run(SERVIDOR_SERVICIOS, SCRIPT_SERVICIOS, [action, service])
    return RedirectResponse(url="/servicios", status_code=303)
```

1. Explicación línea a línea de la integración FastAPI

Línea o bloque	*Explicación funcional*
Líneas 26-32	*Se definen variables de entorno y rutas. Así, el mismo código puede apuntar a distintos servidores sin cambiar la lógica.*
Líneas 34-41	*ssh_run() encapsula la llamada SSH. Evita repetir subprocess.run() en cada ruta y concentra el manejo básico de errores en un solo lugar.*
Línea 35	*El comando final se construye como lista segura de argumentos: ssh, usuario@host, script y argumentos adicionales.*
Líneas 36-38	*Se captura stdout y stderr, importante porque un fallo de systemctl o de SSH podría llegar por cualquiera de los dos canales.*
Líneas 39-40	*Si el comando falla, la función devuelve una cadena prefijada por ERROR. No lanza excepción; devuelve texto consumible por el flujo del MVP.*
Líneas 62-64	*La ruta GET /servicios llama al script remoto con la acción list. Esa es la carga inicial de la página.*
Líneas 66-70	*Se crea una lista de diccionarios. Cada línea de salida del script se divide en nombre y estado usando el separador `*
Líneas 72-75	*Se renderiza servicios.html y se pasa la lista como variable servicios. Jinja2 construirá automáticamente las filas de la tabla.*

Líneas 80-86	***La ruta POST /servicios/accion procesa formularios HTML. Recibe service y action, ejecuta el script remoto y redirige a /servicios.***
Línea 86	***La redirección usa código 303, evitando que al refrescar el navegador se vuelva a enviar el formulario POST anterior.***

2. Por qué usar SSH desde la API

El proyecto utiliza una arquitectura multi-servidor.

En este escenario, la API del panel no necesita estar en la misma máquina que los servicios que administra.

Usar SSH permite mantener esta separación, haciendo el laboratorio más realista:

- *Un servidor expone la interfaz web.*
- *Otro servidor concentra los servicios a controlar.*

Documentación técnica de Docker

1. Objetivo de la solución Docker

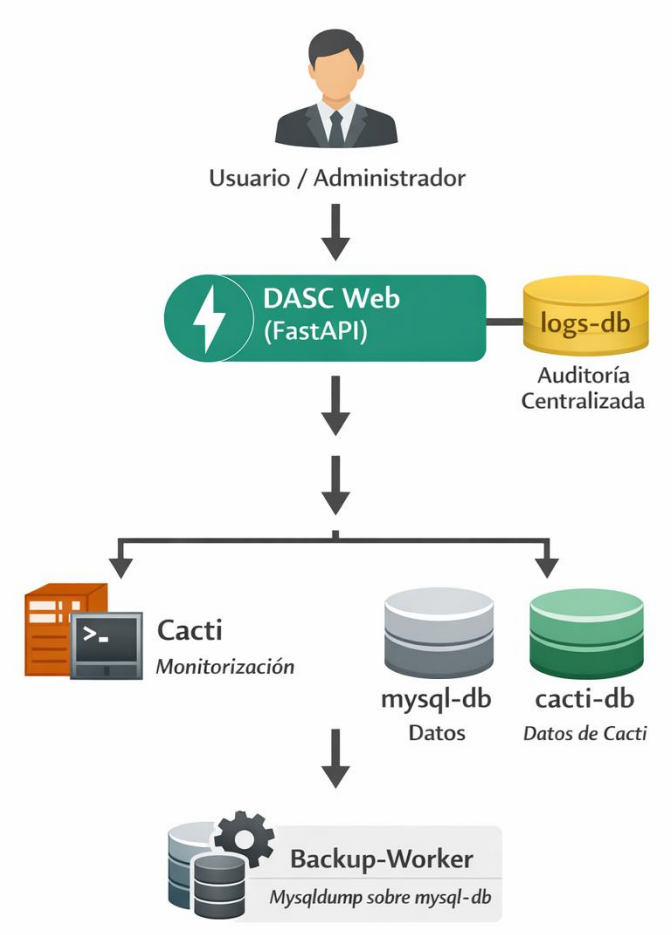
La solución Docker del MVP se diseñó para sustituir el despliegue inicial basado en varias máquinas virtuales y scripts dispersos por una arquitectura reproducible, portable y fácil de levantar con un único fichero docker-compose.yml

El objetivo principal es que cualquier persona con Docker y Docker Compose pueda reconstruir el entorno del proyecto con un proceso controlado y con una separación clara entre panel web, base de datos principal, sistema de copias de seguridad, monitorización y auditoría

La contenerización aporta varias ventajas directas al proyecto. Elimina la dependencia de una OVA concreta, fija la configuración de cada componente en ficheros versionables, permite reflejar la fragmentación del sistema sin obligar a desplegar cinco máquinas virtuales reales y facilita la demostración del MVP en una presentación, porque cada servicio queda identificado por nombre, red, volumen y puerto

En este stack Docker, los servicios no se controlan mediante scripts shell con systemctl ni mediante acceso SSH entre máquinas. El panel DASC usa la API de Docker a través del socket del host para arrancar, parar y reiniciar contenedores. Esa decisión es importante porque cambia el modelo de administración ya no se gestionan procesos del sistema operativo, sino contenedores concretos que representan funciones del MVP

2. Visión general de la arquitectura



La arquitectura final del MVP utiliza seis servicios principales y cuatro redes lógicas. Cada contenedor cumple una función concreta y se comunica solo con los bloques que necesita. Esta decisión reproduce una segmentación similar a la que se tendría en una infraestructura real: zona de exposición (panel), zona de datos, zona de copias de seguridad y zona de monitorización

3. Servicios definidos en docker-compose.yml

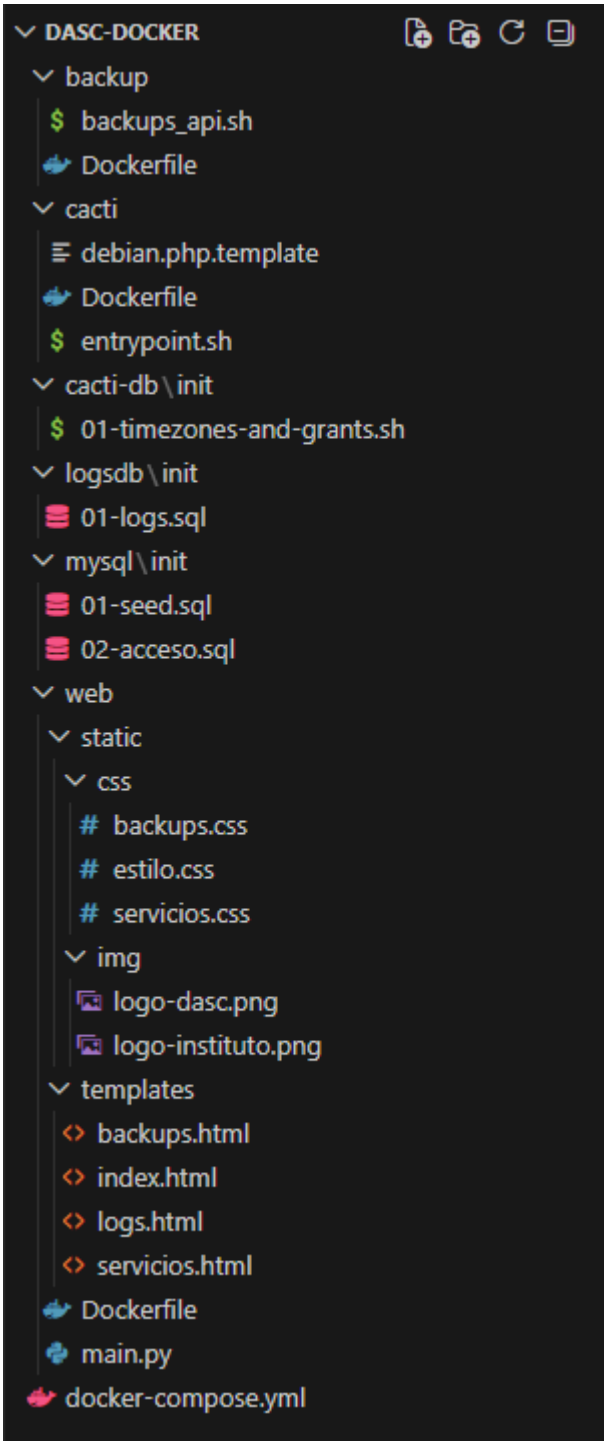
Servicio	Función	Justificación técnica
dasc-web	Panel web y API del MVP	Expone el panel en el puerto 8080, renderiza las plantillas HTML, registra eventos en logs-db y controla contenedores mediante el socket Docker.
mysql-db	Base de datos principal	Aloja dascdb y las bases auxiliares inicializadas desde mysql/init, incluyendo datos empresariales de ejemplo y una base de acceso.
backup-worker	Ejecución de copias	Contenedor de trabajo que ejecuta backups_api.sh y usa mysqldump contra mysql-db.
cacti-db	Base de datos interna de Cacti	Guarda usuarios, settings, plantillas, gráficas y metadatos de la monitorización.
cacti	Monitorización web	Sirve Cacti en el puerto 8081 y se autoconfigura al arrancar mediante entrypoint.sh
logs-db	Auditoría centralizada	Almacena los eventos generados por dasc-web en la tabla eventos, usada por la vista de logs del panel.

4. Redes y volúmenes

El fichero docker-compose.yml no solo define servicios: también define las redes virtuales y los volúmenes persistentes que permiten representar la segmentación del proyecto y conservar los datos entre reinicios.

Elemento	Tipo	Explicación
dmz_net	Red	Zona expuesta donde se coloca dasc-web. Simula el acceso desde el usuario final.
data_net	Red interna	Red marcada como internal: true. Aquí conviven mysql-db, backup-worker, logs-db y el panel cuando necesita acceder a datos.
backup_net	Red interna	Aísla el worker de backups para reflejar una zona separada de copias.
mon_net	Red	Red específica para monitorización, usada por cacti y cacti-db.
db_data	Volumen	Persistencia de /var/lib/mysql en mysql-db.
backups_data	Volumen	Persistencia del directorio /backups del backup-worker.
cacti_db_data	Volumen	Persistencia de MariaDB usada por Cacti.
logs_db_data	Volumen	Persistencia de la base de logs centralizados.

5. Estructura de carpetas del proyecto



web/ Contiene la imagen del panel. Aquí se empaqueta el backend FastAPI, las plantillas Jinja2 y los recursos estáticos. Es la carpeta más visible del MVP porque define la interfaz y la lógica de control de contenedores.

backup/ Agrupa la imagen del worker de backups. Su objetivo no es servir una web, sino ofrecer una herramienta estable desde la que ejecutar mysqldump contra la base principal.

mysql/init/ Contiene scripts SQL de inicialización. Docker los ejecuta automáticamente en el primer arranque de mysql-db, lo que permite que la base ya nazca con datos de ejemplo y con el esquema adicional de acceso.

logsdb/init/ Incluye el SQL que crea la base dasc_logs y la tabla eventos. Es la base de la auditoría del panel.

cacti-db/init/ Incluye un script de inicialización específico para Cacti. Su función es cargar las tablas horarias y dar permisos de lectura al usuario de Cacti sobre la información de zonas horarias de MariaDB.

cacti/ Contiene la imagen personalizada de Cacti. En lugar de depender de configuraciones manuales posteriores, genera el archivo /etc/cacti/debian.php y arranca el servicio preparado para trabajar con cacti-db.

6. Explicación detallada de docker-compose.yml

El fichero docker-compose.yml es el núcleo del despliegue. No solo crea contenedores: define dependencias, redes, puertos, volúmenes y variables de entorno. A continuación se explica su lógica por bloques.

- **version: "3.9":** Indica la sintaxis general del fichero Compose. En versiones recientes de Docker Compose esta línea es opcional y suele mostrar un warning informativo, pero no impide el funcionamiento.
- **services:** Abre el bloque donde se declaran todos los contenedores del proyecto.
- **dasc-web:** Define el servicio principal del panel. Usa una build local desde ./web y expone el puerto 8080 del host al puerto 8000 interno del contenedor.
- **depends_on: mysql-db, backup-worker:** Garantiza orden básico de arranque. No significa que la DB esté lista para aceptar conexiones, pero sí que se inicia antes que el panel.
- **volumes: /var/run/docker.sock:/var/run/docker.sock:** Monta el socket Docker del host dentro del contenedor. Es la pieza que permite a main.py consultar y controlar otros contenedores desde Python.
- **environment de dasc-web:** Inyecta las credenciales y el host de logs-db. Gracias a esto, el panel no hardcodea la conexión a la base de auditoría.
- **mysql-db:** Usa la imagen oficial mysql:8, crea la base dascdb y monta el directorio ./mysql/init para ejecutar los scripts SQL automáticos.
- **backup-worker:** Construye una imagen propia basada en mysql:8 para disponer de mysqldump. Se conecta a data_net y backup_net y monta el volumen persistente de copias.
- **cacti-db:** Usa MariaDB 10.6 y carga scripts de inicialización específicos desde ./cacti-db/init
- **cacti:** Construye una imagen Debian personalizada. Publica el puerto 8081 y se conecta exclusivamente a mon_net
- **logs-db:** Segunda MariaDB dedicada a auditoría. La separación respecto a mysql-db es una decisión arquitectónica deliberada: el proyecto distingue datos funcionales y datos de trazabilidad
- **volumes:** Define nombres de volúmenes persistentes de Docker. Son gestionados por Docker y sobreviven aunque el contenedor se elimine
- **networks:** Define las redes lógicas. Las dos redes data_net y backup_net se marcan como internal: true para evitar exposición innecesaria

7. Carpeta web/: panel FastAPI y control del stack

7.1 Dockerfile de web

La imagen del panel se construye desde python:3.11. Se usa una imagen generalista y estable porque el backend no necesita un sistema operativo completo ni servicios auxiliares; solo requiere Python, librerías web y el cliente Docker SDK.

WORKDIR /app fija el directorio de trabajo del contenedor. Todas las rutas relativas del proyecto se resuelven ahí. Después se copian main.py, templates y static. Esto significa que la imagen final contiene tanto la lógica del backend como los recursos del frontend que el panel renderiza con Jinja2.

El comando pip install añade las dependencias exactas del panel: FastAPI y Uvicorn para servir la aplicación, Jinja2 para renderizar HTML, python-multipart para procesar formularios, docker para controlar contenedores y pymysql para registrar y consultar logs desde logs-db.

```
FROM python:3.11
 WORKDIR /app
 COPY main.py .
 COPY templates ./templates
 COPY static ./static
 RUN pip install --no-cache-dir fastapi uvicorn jinja2 python-multipart
docker pymysql
 EXPOSE 8000
 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7.2 main.py: explicación funcional orientada a Docker

Aunque main.py forma parte de la API del proyecto, dentro de esta memoria solo se explican los bloques relacionados con Docker, logs y backups, porque son los que justifican la contenerización.

- **Bloque de imports:** Importa FastAPI y utilidades web, pero también pymysql y docker. Ese detalle marca la filosofía del proyecto: el panel no es solo una web estática, sino el orquestador de servicios y la pasarela hacia la auditoría
- **Jinja2Templates y StaticFiles:** Permiten que la imagen web sirva plantillas y recursos propios sin depender de Apache o Nginx. Uvicorn responde directamente
- **MANAGED_CONTAINERS:** Lista de contenedores gestionables desde el panel. Es un control de seguridad funcional: el panel no debería poder arrancar o parar cualquier contenedor del host, solo los definidos en esa lista

- **docker_client = docker.DockerClient(...):** Conecta el backend al Docker Engine a través del socket montado. Este punto sustituye al antiguo enfoque con shell scripts y SSH. Aquí el panel habla directamente con la API local de Docker
- **LOGS_DB_*:** Variables de entorno para conectar con logs-db sin hardcodear valores en el código. Permiten mover la configuración al compose
- **log_event():** Inserta registros en la tabla eventos. Esta función convierte el panel en emisor de auditoría real y no solo en interfaz visual
- **middleware log_requests:** Registra automáticamente cada petición HTTP relevante. Clasifica la ruta como acceso, servicio, backup o error y la guarda en logs-db
- **docker_ok():** Comprueba que el socket Docker es accesible. Sin esta prueba, el panel podría intentar acciones sobre servicios sin tener permisos ni conexión al motor de contenedores.
- **docker_ps_services():** Obtiene el estado real de los contenedores administrables. Traduce el estado de Docker a valores legibles para la interfaz: Activo, Inactivo, No existe o ERROR
- **docker_service_action():** Realiza start, stop o restart sobre el contenedor seleccionado. Contiene una protección explícita para no parar dasc-web desde la propia interfaz, ya que eso rompería la administración remota del stack.
- **docker_exec():** Lanza comandos dentro de un contenedor concreto. Este patrón se utiliza para ejecutar el script de copias dentro de backup-worker sin exponer shell remotas ni depender de SSH.
- **run_backup_in_worker():** Construye el comando final para ejecutar backups_api.sh dentro del worker de copias.
- **is_ok():** Normaliza la validación del texto devuelto por el script de backup. Se considera éxito si la salida comienza por OK o contiene expresiones equivalentes
- **Ruta /servicios:** Renderiza la vista de servicios con el estado actualizado de los contenedores administrables
- **Ruta /servicios/accion:** Recibe la acción del formulario y dispara docker_service_action(). Después redirige a la vista para que el usuario vea el nuevo estado.
- **Ruta /backups y endpoints relacionados:** Sirven la interfaz de copias y lanzan el script de backup dentro del worker. Aquí se ve claramente la separación de responsabilidades: el panel no hace el dump por sí mismo, sino que ordena al contenedor especializado que lo haga.

- **Ruta /logs:** En el estado funcional final del MVP, esta vista ya reutiliza logs-db para mostrar los eventos que genera el propio panel. Esto permite enseñar en la presentación una auditoría centralizada real del sistema Dockerizado.

8. Carpeta backup/: contenedor trabajador de copias

La carpeta backup representa una decisión de arquitectura relevante. El proyecto podría haber ejecutado mysqldump directamente desde el panel, pero eso mezclaría demasiadas responsabilidades en el mismo contenedor. En cambio, se crea un worker específico cuya única función es generar backups.

8.1 Dockerfile del worker

```
FROM mysql:8
 WORKDIR /scripts
 COPY backups_api.sh /scripts/backups_api.sh
 RUN chmod +x /scripts/backups_api.sh
 CMD ["tail", "-f", "/dev/null"]
```

La elección de mysql:8 como base tiene sentido porque ya incluye las herramientas cliente necesarias para ejecutar mysqldump. No hace falta instalar un sistema completo ni construir un entorno más pesado.

El CMD final deja el contenedor vivo en segundo plano. Esto es importante porque el panel necesita poder ejecutar comandos dentro de él con docker exec en cualquier momento. Si el contenedor terminará tras el arranque, no existiría proceso donde entrar para lanzar backups_api.sh.

8.2 backups_api.sh explicado paso a paso

- **set -euo pipefail:** Hace que el script falle ante errores, uso de variables no definidas y fallos dentro de tuberías. Es una medida básica de robustez
- **TYPE, DB, DEST, NAME...:** Recoge los parámetros de entrada. El panel le pasa el tipo de copia, la base de datos, la ruta de destino, el nombre de fichero, comprensión, retención y notas
- **Validación de parámetros:** Si faltan tipo, base o nombre, el script devuelve un error 400 legible. Así el frontend puede interpretar que faltan datos
- **mkdir -p "$DEST":** Garantiza que el directorio de copias existe antes de escribir el volcado
- **SAFE_NAME:** Reemplaza plantillas de nombre como YYYYMMDD o HHMM por la fecha y hora reales. Esto permite definir nombres dinámicos desde la interfaz

- **MYSQL_HOST / MYSQL_PORT / MYSQL_USER / MYSQL_PASSWORD:** Toman los valores del entorno. Gracias a esto, el script no depende de una IP hardcodeada; usa el hostname mysql-db que resuelve Docker Compose por DNS interno
- **mysqldump ... --databases "$DB":** Ejecuta la exportación real contra la base indicada
- **Bloque de COMPRESS:** En esta imagen concreta no se usa gzip real, así que se deja una salida informativa y se guarda sin compresión efectiva. Es una limitación reconocida del MVP.
- **Retención con find:** Elimina copias antiguas según los días definidos.
- **echo "OK: Backup ...":** Devuelve un mensaje sencillo y procesable por el backend.

9. Carpeta mysql/init/: inicialización de la base principal

La base principal no se crea vacía. El proyecto aprovecha la característica estándar de las imágenes oficiales MySQL: todo script situado en /docker-entrypoint-initdb.d se ejecuta en el primer arranque del volumen. Esta decisión es fundamental porque evita trabajo manual posterior y permite que el entorno nazca ya preparado para demo, backups y pruebas funcionales.

9.1 01-seed.sql

Crea el usuario dasc con acceso remoto a dascdb y aplica privilegios completos sobre esa base. Después genera un esquema de empresa ejemplo con tablas de oficinas, departamentos, empleados, proyectos, asignaciones, partes de horas y nóminas.

Este bloque cumple dos objetivos. El primero es técnico: tener una base real sobre la que probar mysqldump y validar que los backups no son triviales. El segundo es de demostración: el MVP no muestra una base vacía, sino un entorno con información suficientemente realista para justificar el panel.

9.2 02-acceso.sql

Crea la base dasc_acceso con tablas de usuarios, dispositivos, intentos de acceso y bloqueos. Esta parte es especialmente interesante porque prepara el camino para una evolución del proyecto hacia control de acceso y auditoría de seguridad más rica.

Aunque la visualización actual del MVP se apoya en logs-db y en la tabla eventos, este segundo SQL demuestra una intención de diseño: separar la lógica operativa de los datos específicos de seguridad y acceso.

10. Carpeta logsdb/init/: auditoría centralizada

La base logs-db es una de las decisiones más importantes de la versión Docker. En lugar de mezclar los eventos con la base principal, el proyecto crea una MariaDB independiente dedicada a auditoría. Esta fragmentación mejora claridad, seguridad y posibilidad de crecimiento.

```
CREATE DATABASE IF NOT EXISTS dasc_logs;
 USE dasc_logs;
 DROP TABLE IF EXISTS eventos;
 CREATE TABLE eventos (
 id BIGINT AUTO_INCREMENT PRIMARY KEY,
 fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 origen VARCHAR(40) NOT NULL,
 tipo VARCHAR(40) NOT NULL,
 usuario VARCHAR(60) DEFAULT NULL,
 ip_origen VARCHAR(45) DEFAULT NULL,
 recurso VARCHAR(120) DEFAULT NULL,
 resultado ENUM('OK','ERROR') NOT NULL,
 detalle VARCHAR(255) DEFAULT NULL
 );
```

La tabla eventos responde a una pregunta concreta: qué hizo el sistema, cuándo, desde dónde y con qué resultado. Al usar una sola tabla simple, el MVP facilita la lectura y la visualización en el panel /logs.

El registro inicial "Inicio del sistema" se inserta como semilla para confirmar visualmente que la base de logs se ha inicializado correctamente incluso antes de que el usuario navegue por la interfaz.

11. Carpeta cacti-db/init/ y carpeta cacti/: monitorización contenerizada

Cacti requiere más trabajo que el resto de servicios porque no basta con arrancar Apache y MariaDB. Necesita un archivo de configuración, una base de datos inicializada, rutas de escritura, tablas horarias y un arranque en el orden correcto. Por eso se utiliza una imagen personalizada en lugar de depender solo de la imagen oficial empaquetada.

11.1 cacti-db/init/01-timezones-and-grants.sh

Este script carga las tablas horarias de MariaDB usando mysql_tzinfo_to_sql y después concede al usuario de Cacti permiso de lectura sobre mysql.time_zone_name. La razón es práctica: muchas instalaciones de Cacti fallan o muestran problemas si la base no puede consultar correctamente las zonas horarias.

El script se ejecuta automáticamente en el primer arranque de cacti-db gracias al montaje del directorio ./cacti-db/init en /docker-entrypoint-initdb.d.

11.2 cacti/Dockerfile

La imagen parte de debian:12 e instala Apache, cliente MariaDB, Cacti, rrdtool, SNMP, PHP y cron. Esta elección refleja una necesidad real: Cacti es una aplicación más pesada y más dependiente del sistema que el panel FastAPI, por lo que requiere una imagen más cercana a un servidor tradicional.

El Dockerfile también ajusta los php.ini para aumentar memory_limit, fijar max_execution_time y establecer la zona horaria. Sin estos ajustes, el panel web de Cacti y algunos procesos de inicialización pueden comportarse de forma errática.

11.3 debian.php.template

Este archivo es la clave para evitar configuraciones manuales posteriores. En lugar de editar /etc/cacti/debian.php a mano dentro del contenedor, se usa una plantilla con placeholders. El entrypoint la rellena con las variables CACTI_DB_HOST, CACTI_DB_NAME, CACTI_DB_USER y CACTI_DB_PASS en cada arranque.

11.4 entrypoint.sh, explicado paso a paso

- **Variables CACTI_DB_* y TZ:** Definen valores por defecto para que el contenedor no falle aunque alguna variable no se inyecte.
- **ServerName localhost:** Elimina el warning típico de Apache sobre el nombre del servidor.
- **Generación de /etc/cacti/debian.php:** Convierte la plantilla en configuración real usando sed. Así Cacti siempre apunta al servicio cacti-db de Docker y no a localhost.

- **Bucle de espera a la base:** Usa mariadb-admin ping en un bucle de hasta 60 intentos. Esta parte evita arrancar Cacti antes de que la base esté preparada
- **Detección de cacti.sql:** Busca el esquema SQL en varias rutas posibles. Esto hace el contenedor más portable frente a diferencias de empaquetado
- **HAS_TABLES:** Comprueba si la base de datos de Cacti ya tiene tablas. Si está vacía, importa el schema automáticamente
- **Actualización del usuario admin:** Fuerza una contraseña conocida para el usuario admin del instalador. Es una decisión orientada a la demo del MVP
- **REPLACE INTO settings url_path:** Ajusta la ruta base de Cacti a /cacti/. Esta línea fue necesaria para evitar problemas de redirección y mantener la aplicación accesible bajo el puerto 8081 con subruta
- **Corrección de sesiones PHP:** Crea y ajusta permisos de /var/lib/php/sessions para prevenir problemas de login
- **Corrección de rutas escribibles:** Crea y asigna permisos a los directorios internos que Cacti necesita para consultas SNMP, scripts y secretos CSRF
- **service cron start + apache2ctl -D FOREGROUND:** Activa el poller y deja Apache en primer plano, que es la forma correcta de mantener vivo el contenedor

12. Comandos de trabajo: construcción, arranque y verificación

Esta sección resume el flujo operativo recomendado. Se incluyen comandos pensados para Windows con Docker Desktop y también válidos conceptualmente para Ubuntu, adaptando solo la shell.

12.1 Preparación inicial

```
cd C:\Users\colme\Documents\dasc-docker
 # o en Ubuntu
 cd ~/dasc-docker
docker compose up -d --build
```

Este comando construye las imágenes locales (web, backup y cacti) y levanta todos los servicios en segundo plano. Se recomienda en el primer arranque o tras cambios en Dockerfiles, scripts o main.py.

12.2 Comprobación del estado

```
docker compose ps
 docker ps --format "table {{.Names}}   {{.Status}} {{.Ports}}"
```

Permite confirmar que los seis servicios principales están en estado Up y que los puertos 8080 y 8081 están publicados correctamente.

12.3 Accesos funcionales

```
Panel DASC: http://localhost:8080/
 Cacti: http://localhost:8081/cacti/
```

Si se usa una máquina virtual con adaptador puente o IP de red local, estas rutas se sustituyen por la IP de la VM.

12.4 Rebuild selectivo

```
docker compose up -d --build dasc-web
 docker compose up -d --build cacti
```

Muy útil cuando solo cambia la web o solo cambia el servicio de monitorización. Evita reconstrucciones completas innecesarias.

12.5 Logs y depuración

```
docker compose logs -f dasc-web
 docker compose logs -f cacti
 docker compose logs -f mysql-db
```

En un stack Docker, estos logs sustituyen a la revisión clásica de journalctl o systemctl status en el host. Son la primera fuente para localizar problemas de importación SQL, errores de Python o fallos de inicialización de Cacti.

12.6 Parada y limpieza

```
docker compose down
 # Parada y eliminación de volúmenes (destructivo)
 docker compose down -v
```

down detiene y elimina contenedores y redes temporales, pero mantiene los volúmenes. down -v borra también los volúmenes persistentes, por lo que obliga a reejecutar todos los init scripts SQL desde cero.

13. Flujo de funcionamiento del stack, de punta a punta

1. El usuario abre el panel en el puerto 8080. Docker redirige la petición al puerto 8000 interno de dasc-web
2. Uvicorn entrega la ruta solicitada y renderiza la plantilla correspondiente
3. El middleware de main.py registra la petición en logs-db mediante la función log_event()
4. Si el usuario entra en Servicios, dasc-web consulta la API de Docker y recupera el estado real de mysql-db, backup-worker y cacti
5. Si el usuario pulsa Start, Stop o Restart, dasc-web llama a docker_service_action(), que actúa sobre el contenedor objetivo
6. Si el usuario ejecuta una copia, dasc-web no hace el dump directamente: entra en backup-worker con docker exec y ejecuta backups_api.sh
7. backup-worker usa mysqldump sobre el hostname mysql-db, que Docker resuelve internamente gracias a data_net
8. El fichero SQL queda almacenado en el volumen backups_data, persistente fuera del ciclo de vida del contenedor
9. Si el usuario entra en Logs, dasc-web consulta la tabla eventos de logs-db y muestra la auditoría ya consolidada en una vista HTML
10. Si el usuario abre Cacti, el navegador accede al puerto 8081, que redirige al Apache interno del contenedor cacti, configurado por entrypoint.sh y respaldado por cacti-db.

14. Justificación técnica de las decisiones de diseño

- **Separar mysql-db y logs-db:** Evita mezclar datos operativos con auditoría. Hace la arquitectura más limpia y más defendible en la memoria del proyecto
- **Usar backup-worker dedicado:** Permite encapsular las copias y no sobrecargar el contenedor web con herramientas de base de datos
- **Montar /var/run/docker.sock en dasc-web:** Es la decisión que sustituye el antiguo sistema de scripts shell y SSH. Da al panel la capacidad de actuar como consola de control del stack.
- **Mantener Cacti en contenedor independiente:** Aísla una aplicación pesada y compleja. Además, evita depender de una instalación previa en el host.
- **Inicialización automática con directorios init:** Convierte el proyecto en reproducible. Un entorno vacío puede quedar listo con un solo docker compose up -d --build
- **Redes internas:** Simulan zonas separadas y reducen la exposición innecesaria
- **Volúmenes persistentes:** Permiten que el MVP sobreviva a recreaciones de contenedores sin perder bases de datos ni backups

15. Incidencias reales y consideraciones prácticas

- **Diferencias entre Windows y Linux**
  Durante el desarrollo hubo fricción al ejecutar comandos complejos con PowerShell, especialmente con docker exec, comillas y redirecciones. En Linux o dentro de una VM Ubuntu el comportamiento fue mucho más estable. Para uso diario y depuración profunda, la VM Linux es el entorno recomendado.
- **Confusión entre Cacti del host y Cacti Docker**
  En una fase del proyecto coexistieron una instalación previa de Cacti en el host y la versión contenerizada. Esto provocó accesos aparentemente correctos que en realidad no salían del contenedor. La lección técnica es clara: en un despliegue Docker serio conviene evitar instalaciones paralelas del mismo servicio en el host.
- **La ruta y sesiones de Cacti** requirió más ajustes que el resto de componentes. La solución final consistió en centralizar la configuración dentro de entrypoint.sh y debian.php.template en lugar de depender de modificaciones manuales tras el arranque.
- **Zona horaria de logs**

  La tabla eventos puede reflejar hora del contenedor y no siempre la hora visual del cliente. Esto debe mencionarse en la memoria como efecto normal de la separación entre host, contenedor y base de datos.

- **Dependencia del socket Docker**

  El panel necesita acceso a /var/run/docker.sock para controlar contenedores. Si ese volumen no está montado o si Docker no es accesible, la funcionalidad de servicios y backups no opera correctamente.

16. Conclusión

La parte Docker del MVP de DASC Server Manager está concebida como una mini infraestructura segmentada. En lugar de levantar una sola aplicación monolítica, el proyecto reparte funciones entre varios contenedores especializados: panel, base de datos principal, worker de copias, auditoría centralizada y monitorización

La clave técnica del resultado final es la automatización: docker-compose.yml define el stack; los directorios init crean bases de datos y datos de prueba; y los entrypoints resuelven la configuración compleja de Cacti sin depender de intervención manual posterior

Desde el punto de vista académico y de presentación, esta arquitectura permite defender varias decisiones importantes: fragmentación, separación de responsabilidades, persistencia mediante volúmenes, redes internas y reproducibilidad del despliegue. Eso hace que la solución no sea solo funcional, sino también justificable a nivel de diseño

Resumen fichero por fichero

Archivo	Papel dentro del stack Docker
docker-compose.yml	Orquesta todos los servicios, puertos, redes, volúmenes y variables de entorno
web/Dockerfile	Construye la imagen del panel FastAPI
web/main.py	Controla la interfaz, la auditoría en logs-db, la gestión de servicios y la ejecución de backups
backup/Dockerfile	Construye la imagen del worker de backups
backup/backups_api.sh	Realiza el mysqldump, aplica retención y devuelve el resultado
mysql/init/01-seed.sql	Crea la base de datos empresarial de ejemplo y el usuario dasc
mysql/init/02-acceso.sql	Crea una base adicional orientada a control de acceso
logsdb/init/01-logs.sql	Crea dasc_logs y la tabla eventos
cacti-db/init/01-timezones-and-grants.sh	Carga zonas horarias y permisos para Cacti
cacti/Dockerfile	Construye una imagen Debian con Cacti, Apache, PHP, cron y herramientas auxiliares
cacti/debian.php.template	Plantilla del archivo de configuración principal de Cacti
cacti/entrypoint.sh	Configurar Cacti al arrancar, espera a la base, importa schema y lanza Apache
web/templates/*.html	Plantillas del panel. Se documentan en la memoria de frontend, no en detalle aquí
web/static/css/*.css	Hojas de estilo del panel. Se documentan en la memoria de frontend, no en detalle aquí

Documentación técnica de Backups

1. Contexto y alcance del módulo de backups

El módulo de backups del MVP se ha diseñado para resolver una necesidad muy concreta del proyecto: permitir que el usuario ejecute copias de seguridad de la base de datos desde una interfaz web local, con un flujo sencillo, comprensible y controlado desde la infraestructura propia del cliente. La prioridad del MVP no ha sido construir todavía un sistema avanzado de restauración, descarga o incremental real, sino demostrar que el producto es capaz de lanzar copias reales, guardarlas en otra máquina y hacerlo a través de una experiencia de usuario coherente.

2. Situación del módulo dentro del proyecto DASC Server Manager

DASC Server Manager nace como un software local orientado a empresas que quieren controlar sus datos sin depender de terceros. En la primera entrega se definió como objetivo la implementación de un sistema completo de copias automatizadas y, en el alcance del MVP, se dejó expresamente indicada la capacidad para realizar copias completas. En la versión definitiva del producto se reservaron la restauración, la descarga, la visualización del historial, la eliminación y el soporte completo para copias incrementales y diferenciales

Por esa razón, la solución finalmente implementada en el MVP se centra en un backup completo real, ejecutado desde una interfaz web, apoyado por una API intermedia y un script Bash remoto.

La interfaz ya incorpora los tres botones y sus formularios para dejar preparada la evolución del producto, pero internamente el motor del MVP sigue una lógica de dump completo, porque esa era la parte imprescindible para demostrar valor funcional y estabilidad en la entrega.

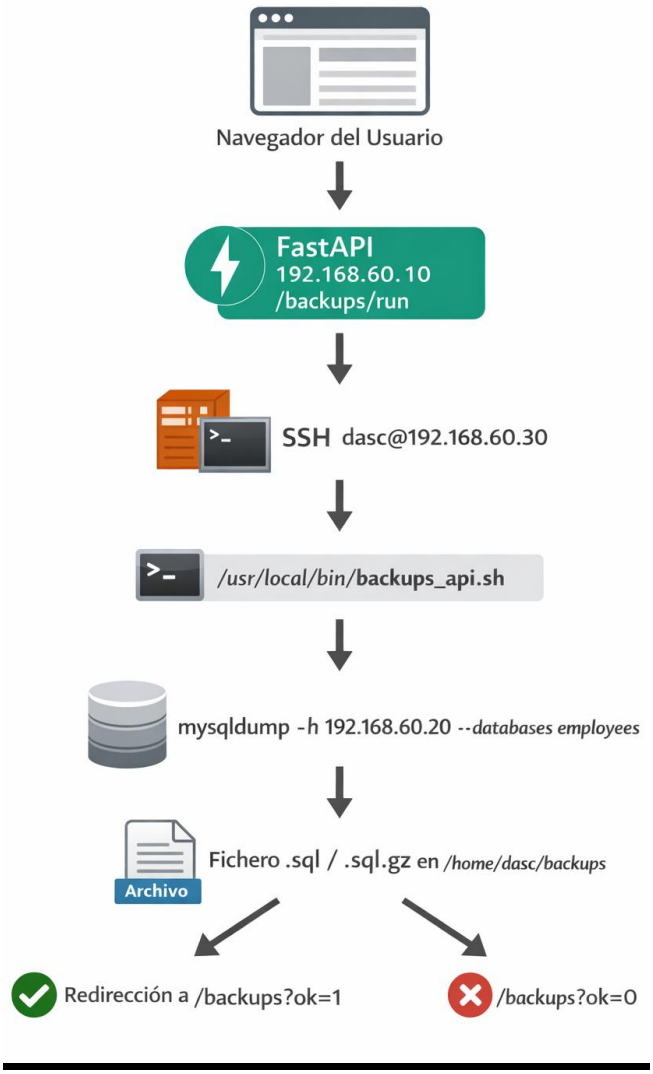
3. Arquitectura real por máquinas

El módulo de backups no se ejecuta todo en una sola máquina. Se ha separado por funciones para acercarse a una arquitectura real y para no mezclar el panel de gestión con la base de datos ni con el almacenamiento de copias. Esta separación además simplifica la seguridad y permite explicar el producto de forma más profesional.

Máquina	IP	Rol principal	Elementos relacionados con backups
Panel-API	192.168.60.10	Interfaz web + Fast API	Recibe el formulario, valida datos, conecta por SSH con la máquina de backups y redirige el resultado al usuario
BaseDeDatos	192.168.60.20	MariaDB	Contiene la base de datos real que se copia. Atiende conexiones remotas de mysqldump desde la máquina de backups
Backups	192.168.60.30	Ejecución y almacenamiento	Guarda el script /usr/local/bin/backups_api.sh, almacena las copias en /home/dasc/backups y usa credenciales de MariaDB para extraer el dump

Esta arquitectura permite un flujo claro: el usuario pulsa un botón en el navegador, la API del panel recibe la petición, la API no hace el dump localmente sino que se conecta por SSH a la máquina 192.168.60.30, y allí un script Bash ejecuta mysqldump contra la base MariaDB situada en 192.168.60.20. De esta manera las copias no se guardan en la misma máquina que la base de datos y se mantiene una separación lógica entre gestión, extracción y almacenamiento.

Flujo lógico del módulo de backups



4. Estructura de carpetas, ficheros y responsabilidades

Para que la documentación de backups no pise otras memorias, conviene separar muy bien qué fichero pertenece a qué parte. En este módulo hay cuatro piezas principales: la plantilla HTML del apartado de copias, su hoja de estilos específica, el backend FastAPI y el script Bash remoto.

Servidor	Ruta o fichero	Función
Panel-API	main.py	Contiene las rutas /backups y /backups/run y la función ssh_run para lanzar el script remoto.
Panel-API	templates/backups.html	Interfaz de backups con modales para completo, incremental y diferencial
Panel-API	static/css/backups.css	Estilo visual del módulo de copias y del mensaje de éxito/error
Panel-API	config.env	Variables de entorno leídas por el servicio systemd: usuario SSH, host de backups y URL de Cacti
Backups	/usr/local/bin/backups_api.sh	Script Bash que valida parámetros, genera el dump, comprime y aplica retención
Backups	/home/dasc/backups	Directorio de almacenamiento de copias del MVP
Backups	/home/dasc/.my.cnf	Credenciales del usuario de MariaDB empleadas por mysqldump
BaseDeDatos	/etc/mysql/mariadb.conf.d/50-server.cnf	Configuración de MariaDB. En pruebas avanzadas se tocó también para habilitar binlogs, aunque esa parte queda fuera del MVP final.

4.1 Motivo de cada elemento

main.py existe porque el navegador no debe ejecutar directamente comandos de sistema ni conectarse por SSH. Se necesita una capa intermedia que reciba formularios HTTP y controle la lógica del proceso. El HTML y el CSS existen para que la parte de copias tenga una estética coherente con el resto del panel y para que el usuario no tenga que memorizar comandos. El script Bash existe para desacoplar la lógica de sistema del backend Python: así la API solo envía parámetros y el servidor de backups ejecuta la operación real.

El directorio /home/dasc/backups se eligió después de una incidencia real con permisos en /var/backups/dasc. El cambio de ruta permitió mantener el MVP operativo sin depender de reglas adicionales de sudo y simplificó mucho la demostración. El fichero /home/dasc/.my.cnf también fue una decisión práctica: evitó incrustar usuario y contraseña de MariaDB en cada comando o en el propio script.

5. Decisiones técnicas del MVP

La implementación final del MVP no es casual; cada decisión se tomó para reducir complejidad sin perder funcionalidad real. En este apartado se justifica el porqué de cada tecnología y de cada simplificación.

Decisión	Motivo técnico y funcional
FastAPI como capa intermedia	Permite exponer páginas HTML y endpoints POST en el mismo servicio, manejar formularios y redirigir resultados sin obligar al usuario a usar consola
SSH entre la API y la máquina de backups	Mantiene la separación por roles y evita instalar toda la lógica de copias en el servidor del panel
Script Bash para la ejecución	Facilita el uso de comandos del sistema, la gestión de argumentos y futuras ampliaciones sin reescribir la API
mysqldump como motor del MVP	Es la herramienta más directa para una copia completa real de la base de datos y encaja con un prototipo funcional.

GZIP opcional	Permite reducir tamaño cuando el usuario lo solicita, manteniendo una opción sin compresión para depuración o inspección
Retención por find -mtime	Es una limpieza simple, entendible y suficiente para un MVP
PRG (Post/Redirect/Get)	Evita reenviar formularios al refrescar y permite mostrar un aviso visual claro en la interfaz
/home/dasc/backups en lugar de /var/backups/dasc	Se eligió por una incidencia real de permisos. Garantiza ejecución con el usuario SSH del proyecto

6. Implementación paso a paso

A continuación se describe el proceso de construcción en el mismo orden lógico en que se fue resolviendo: primero la base de datos remota, después el servidor que ejecuta y almacena las copias, luego la API del panel y finalmente la interfaz web.

6.1 Preparación de MariaDB en la máquina 192.168.60.20

La base de datos del proyecto no está en la máquina de backups, sino en la máquina BaseDeDatos. Por tanto, el dump no puede trabajar por socket local en la .30; necesita conectarse por TCP a la .20. Para ello se configuró MariaDB y se creó un usuario dedicado a backups.

Creación del usuario de backups en MariaDB

```
sudo mariadb
CREATE USER 'dasc_backup'@'192.168.60.30' IDENTIFIED BY 'dasc';
GRANT SELECT, SHOW VIEW, TRIGGER, LOCK TABLES ON employees.* TO
'dasc_backup'@'192.168.60.30';
FLUSH PRIVILEGES;
EXIT;
```

Se eligió un usuario específico en lugar de usar root por dos motivos: seguridad y claridad documental. El usuario de backup solo necesita permisos de lectura y de consulta estructurados sobre la base que se va a copiar, no privilegios totales de administración.

Durante el análisis para la versión final también se activaron los binary logs en /etc/mysql/mariadb.conf.d/50-server.cnf y se comprobó su funcionamiento con SHOW BINARY LOGS;, pero esa línea de trabajo se deja como evolución futura y no forma parte del comportamiento final del MVP.

6.2 Preparación de la máquina 192.168.60.30

La máquina Backups es la que realmente genera los ficheros. En ella se instaló el cliente necesario para disponer de mysqldump, se creó la carpeta de destino, se guardaron las credenciales de MariaDB y se instaló el script de ejecución.

Instalación del cliente en la máquina Backups

```
sudo apt update
sudo apt install -y
mysql-client
```

El paquete mysql-client fue necesario porque inicialmente mysqldump no existía en la máquina de backups. Sin esta herramienta el script podía crear un fichero vacío pero no generar contenido real.

Preparación del directorio y de las credenciales del usuario dasc

```
mkdir -p /home/dasc/backups
cat > ~/.my.cnf<<'EOF'
[client]
user=dasc_backup
password=dasc
host=192.168.60.20
EOF
chmod 600 ~/.my.cnf
```

El fichero ~/.my.cnf se guardó en el HOME del usuario dasc porque la API conecta por SSH justamente con ese usuario. Esta precisión fue importante: en una prueba intermedia se llegó a comprobar un .my.cnf en el HOME de otro usuario, y por eso mysqldump seguía intentando conectar como dasc sin contraseña.

Creación y activación del script remoto

```
sudo nano /usr/local/bin/backups_api.sh
sudo chmod +x /usr/local/bin/backups_api.sh
```

El script se ubicó en /usr/local/bin porque es una ruta estándar para utilidades de administración locales y facilita su invocación desde SSH sin necesidad de navegar por carpetas de proyecto.

6.3 Preparación de la API del panel en 192.168.60.10

La API del panel se ejecuta como servicio systemd y expone tanto páginas HTML como acciones POST. Para backups fueron necesarias dos piezas: las variables de entorno y la ruta /backups/run.

config.env usado por el servicio de la API

```
config.env ×
API > api-automatizacion > config.env
1   SSH_USER=dasc
2
3   SERVICIOS_HOST=192.168.60.30
4   BACKUPS_HOST=192.168.60.30
5
6   CACTI_URL=/cacti/
```

La variable más importante para el módulo de backups es BACKUPS_HOST, que apunta a 192.168.60.30. SSH_USER también es clave porque determina con qué cuenta se abre la conexión remota y, por tanto, qué HOME se utiliza y qué credenciales de MariaDB estarán disponibles.

La ruta /backups/run recibe los datos del formulario, valida el tipo de backup, monta el array de argumentos y llama a la función ssh_run. Después no devuelve directamente un JSON al usuario sino una redirección a /backups?ok=1 o /backups?ok=0. Este patrón simplifica mucho la UX.

6.4 Construcción de la interfaz de usuario

La parte visual se implementó en un template propio, backups.html, y en una hoja de estilos específica, backups.css. Se reutilizó la estética global del panel para no crear una pantalla aislada o incoherente respecto al resto del MVP.

La pantalla muestra una cabecera, un menú lateral, una zona principal con tarjetas y tres modales CSS-only: uno para backup completo, otro para incremental y otro para diferencial. Cada modal envía sus datos a /backups/run mediante método POST. Además, tras integrar el flujo de redirección, se añadió un banner visual verde o amarillo para indicar si la operación terminó correctamente

7. Explicación línea a línea del backend FastAPI

En este apartado se documenta únicamente la parte de main.py que afecta al módulo de backups. No se desarrolla aquí la lógica de logs ni la de servicios salvo para contextualizar el fichero

Variables de entorno y constantes

```
USUARIO = os.getenv("SSH_USER", "dasc")
SERVIDOR_BACKUPS = os.getenv("BACKUPS_HOST", "192.168.60.30")
SERVIDOR_SERVICIOS = os.getenv("SERVICIOS_HOST", "192.168.60.30")
CACTI_URL = os.getenv("CACTI_URL", "http://127.0.0.1/cacti/")

SCRIPT_SERVICIOS = "/usr/local/bin/servicios_api.sh"
SCRIPT_BACKUPS = "/usr/local/bin/backups_api.sh"
```

USUARIO define el nombre con el que la API abrirá la sesión SSH. SERVIDOR_BACKUPS indica la IP de la máquina donde se ejecuta el script remoto. SCRIPT_BACKUPS fija la ruta exacta del ejecutable. Se usan valores por defecto para que la aplicación siga arrancando incluso si falta alguna variable, aunque lo normal es que el servicio systemd cargue config.env.

Función ssh_run

```
def ssh_run(host: str, script: str, args: list[str]) -> str:
    cmd = ["ssh", f"{USUARIO}@{host}", script] + args
    res = subprocess.run(cmd, capture_output=True, text=True)
    out = (res.stdout or "").strip()
    err = (res.stderr or "").strip()
    if res.returncode != 0:
        return f"ERROR ({res.returncode}): {err or out}"
    return out
```

Esta función encapsula toda la ejecución remota. Primero construye el comando ssh con usuario, host, ruta del script y argumentos. Después lanza subprocess.run capturando stdout y stderr para poder inspeccionar errores. Si el código de retorno es distinto de cero, no lanza excepción sino que devuelve una cadena que empieza por ERROR. Esta decisión facilita la lógica posterior porque la ruta /backups/run solo tiene que comprobar si la salida comienza por ese prefijo.

Ruta GET /backups

```
@app.get("/backups")
def backups(request: Request):
    ok = request.query_params.get("ok")
    return templates.TemplateResponse("backups.html", {"request": request, "ok": ok})
```

Esta ruta sirve la pantalla del módulo y, además, lee el parámetro ok de la query string. Gracias a esto, después de ejecutar una copia, la interfaz puede mostrar un aviso visual de éxito o error sin necesidad de JavaScript adicional.

Ruta POST /backups/run

```
@app.post("/backups/run")
def backups_run(
    type: str = Form(...),
    db: str = Form(...),
    dest: str = Form("/backups"),
    name: str = Form(...),
    compress: str = Form("none"),
    retention: int = Form(7),
    base_ref: str = Form(""),
    notes: str = Form("")
):
    if type not in ["full", "incremental", "differential"]:
        return RedirectResponse(url="/backups?ok=0&err=type", status_code=303)

    dest = "/backups"

    if compress not in ["none", "gzip"]:
        compress = "none"

    args = [type, db, dest, name, compress, str(retention), base_ref, notes]
    out = run_backup_in_worker(args)

    ok = 1 if is_ok(out) else 0
    return RedirectResponse(url=f"/backups?ok={ok}&msg={quote(out)}", status_code=303)
```

type recoge el tipo de copia. db recibe el nombre de la base. dest define la carpeta remota donde se guardará. name es el nombre deseado del fichero. compress y retention controlan la compresión y la limpieza. base_ref y notes se aceptan porque la interfaz ya los presenta, aunque en el MVP no alteran el motor real del backup.

La validación del tipo evita que se invoquen valores arbitrarios desde el navegador. Después se construye la lista de argumentos exactamente en el orden esperado por el script Bash. La variable ok se calcula simplemente comprobando si la salida no empieza por ERROR. Finalmente se usa RedirectResponse con código 303 para volver a la pantalla /backups y evitar problemas de reenvío de formulario.

8. Explicación línea a línea del script Bash de backups

El script Bash es el corazón operativo del módulo. Se ejecuta en la máquina 192.168.60.30 y se encarga de recibir parámetros, preparar el nombre del fichero, llamar a mysqldump, comprimir si hace falta y limpiar ficheros antiguos.

```bash
#!/bin/bash
set -euo pipefail

TYPE="${1:-}"
DB="${2:-}"
DEST="${3:-/var/backups/dasc}"
NAME="${4:-}"
COMPRESS="${5:-gzip}"
RETENTION="${6:-7}"
BASE_REF="${7:-}"
NOTES="${8:-}"

if [[ -z "$TYPE" || -z "$DB" || -z "$NAME" ]]; then
  echo "ERROR (400): Faltan parámetros. Uso: backups_api.sh <type> <db> <dest> <name> <compress> <retention> <base_ref> <notes>"
  exit 1
fi

mkdir -p "$DEST"
DATE="$(date +'%Y%m%d-%H%M%S')"

SAFE_NAME="$NAME"
SAFE_NAME="${SAFE_NAME//YYYYMMDD/$(date +'%Y%m%d')}"
SAFE_NAME="${SAFE_NAME//HHMM/$(date +'%H%M')}"
SAFE_NAME="${SAFE_NAME//YYYYMMDD-HHMM/$(date +'%Y%m%d-%H%M')}"

OUTFILE="$DEST/$SAFE_NAME"

MYSQL_HOST="192.168.60.20"
mysqldump -h "$MYSQL_HOST" --protocol=tcp --single-transaction --databases "$DB" > "$OUTFILE"

if [[ "$COMPRESS" == "gzip" ]]; then
  gzip -f "$OUTFILE"
  OUTFILE="$OUTFILE.gz"
fi

if [[ "$RETENTION" != "0" ]]; then
  find "$DEST" -type f -mtime +"$RETENTION" -name "*.sql*" -delete || true
fi

echo "OK: Backup $TYPE creado en $OUTFILE"
```

8.1 Encabezado y seguridad básica del script

La primera línea, #!/bin/bash, indica que el script se ejecutará con Bash. La instrucción set -euo pipefail obliga a abortar ante errores, uso de variables no definidas o fallos en tuberías. Esta práctica es importante porque evita que el script siga adelante en silencio y devuelva un falso OK cuando en realidad algo falló a mitad del proceso.

8.2 Recepción de argumentos

TYPE, DB, DEST, NAME, COMPRESS, RETENTION, BASE_REF y NOTES se rellenan a partir de la posición de cada argumento. La sintaxis ${1:-} o ${3:-/var/backups/dasc} permite manejar casos donde el valor no llega: si falta TYPE, por ejemplo, la variable quedará vacía; si falta DEST, se aplicará una ruta por defecto.

Aunque BASE_REF y NOTES no modifican todavía la lógica de backup, se aceptan para no romper la interfaz y para dejar preparado el terreno de la versión final.

8.3 Validación de mínimos obligatorios

El bloque if [[-z "$TYPE" || -z "$DB" || -z "$NAME"]]; then ... valida que al menos haya tipo, base y nombre. Si alguno falta, el script imprime un mensaje de error con código 400 y termina con exit 1. Esto es importante porque evita crear archivos ambiguos o lanzar mysqldump sin saber qué base se pretende copiar.

8.4 Creación de carpeta y construcción del nombre seguro

mkdir -p "$DEST" crea la carpeta de destino si no existe y no falla si ya estaba creada. SAFE_NAME toma el nombre enviado por la interfaz y sustituye los placeholders YYYYMMDD y HHMM por la fecha y la hora reales. Gracias a esto el usuario puede escribir un patrón legible como backup-completo-YYYYMMDD.sql y dejar que el sistema lo convierta en un nombre único.

8.5 Línea de extracción de datos

La línea crítica es la de mysqldump. Se fija MYSQL_HOST=192.168.60.20 y se usa --protocol=tcp para evitar que la herramienta intente conectarse por socket local en la .30. --single-transaction permite un dump más seguro para tablas InnoDB, reduciendo problemas de consistencia durante la lectura.

Además, el usuario y la contraseña no aparecen en la línea porque mysqldump los toma de /home/dasc/.my.cnf. Esta decisión mejora la limpieza del script y reduce exposición de credenciales.

8.6 Compresión y retención

Si COMPRESS vale gzip, el script comprime el dump con gzip -f y actualiza OUTFILE para reflejar que ahora la copia termina en .gz. Después, si RETENTION es distinto de 0, se ejecuta find sobre la carpeta de destino y se borran los ficheros *.sql* con más días de antigüedad que el valor seleccionado.

El criterio elegido para la retención es deliberadamente simple. No se ha creado todavía una base de datos de metadatos ni una política compleja de ciclos. Para un MVP es más importante que la regla sea legible y fácil de demostrar.

9. Flujo completo de ejecución desde la web hasta el fichero final

El comportamiento del módulo de backups se puede explicar como una cadena de pasos consecutivos. Esta secuencia ayuda mucho en la defensa oral porque deja ver que el sistema no es solo una maqueta visual

Paso	Qué ocurre	Dónde ocurre
1	El usuario entra en /backups y abre el modal del tipo de copia que desea ejecutar	Navegador + Panel-API
2	El formulario envía por POST los campos type, db, dest, name, compress, retention, base_ref y notes	Navegador → FastAPI
3	FastAPI valida el tipo y construye el array de argumentos	Panel-API 192.168.60.10
4	La función ssh_run abre una conexión SSH como dasc a 192.168.60.30 y ejecuta /usr/local/bin/backups_api.sh	Panel-API → Backups
5	El script crea la carpeta si hace falta, resuelve el nombre y lanza mysqldump contra 192.168.60.20	Backups 192.168.60.30
6	MariaDB entrega los datos y el dump se guarda en /home/dasc/backups.	BaseDeDatos → Backups

7	El script comprime y limpia antiguas si corresponde, devuelve un texto OK o ERROR	Backups
8	La API interpreta la salida y redirige al usuario a /backups?ok=1 o /backups?ok=0	Panel-API
9	La plantilla muestra un banner visual con el resultado	Navegador

10. Pruebas de validación realizadas

Durante el montaje se realizaron pruebas de conectividad, de servicio, de ejecución remota y de validación funcional. Se recogen aquí porque forman parte de la documentación técnica y justifican que el módulo no solo está programado, sino probado.

Prueba de conectividad SSH con la máquina de backups

```
ssh dasc@192.168.60.30 "hostname && date"
```

Esta prueba se utilizó para confirmar que la API del panel podía llegar correctamente a la .30. Antes de intentar lanzar el script conviene validar siempre conectividad y usuario.

Prueba del endpoint real /backups/run

```
curl -i -X POST http://127.0.0.1:8000/backups/run -d "type=full" -d "db=employees" -d "dest=/home/dasc/backups" -d "name=backup-completo-YYYYMMDD.sql" -d "compress=gzip" -d "retention=30" -d "notes=prueba"
```

La respuesta correcta de esta prueba es una redirección 303 a /backups?ok=1. Ese resultado demostró que el circuito completo, desde FastAPI hasta el script remoto, estaba operativo.

Validación directa de mysqldump desde la máquina de backups

```
ssh dasc@192.168.60.30 "mysqldump --protocol=tcp --databases employees 2>&1 | head -n 10"
```

Con esta prueba se confirmó que el usuario de MariaDB, el fichero ~/.my.cnf y la conectividad hacia la .20 estaban correctamente configurados.

Comprobación del fichero final

```
ssh dasc@192.168.60.30 "ls -lah /home/dasc/backups | tail -n 10"ssh dasc@192.168.60.30 "zcat /home/dasc/backups/test.sql.gz | head -n 20"
```

La primera línea sirve para validar que el archivo existe y tiene tamaño real. La segunda permite visualizar las primeras líneas del dump comprimido sin necesidad de descargarlo.

11. Incidencias reales y resolución

Una parte importante de esta memoria consiste en documentar incidencias reales. No solo muestran dificultades, también justifican decisiones finales del diseño. A continuación se recogen las más relevantes del módulo de backups.

Incidència	Síntoma observado	Causa detectada	Solución aplicada
Servicio FastAPI caído	curl a 127.0.0.1:8000 devolvía conexión rehusada	El servicio systemd no arrancaba porque la ruta de EnvironmentFile apuntaba a un config.env inexistente	Se corrigió la ruta en dasc-api.service, se recargó systemd y se reinició el servicio
Comprobación en la máquina equivocada	Se buscaba /var/backups/dasc en Panel-API y no exista	Las copias se generan en 192.168.60.30, no en 192.168.60.10	Se unificó la comprobación siempre mediante ssh hacia la máquina Backups
Permiso denegado en /var/backups/dasc	El debug devolvía mkdir: permiso denegado	El usuario dasc no tenía permisos de escritura en /var/backups	Para el MVP se cambió la ruta por /home/dasc/backups y se actualizó la interfaz
mysqldump no encontrado	El archivo de salida se creaba vacío o la shell devolvía orden no encontrada	La máquina Backups no tenía instalado el cliente	Se instaló mysql-client en la .30
Conexión por socket local	mysqldump devolvía error 2002 sobre /var/run/mysqld/mysqld.sock	La base estaba en 192.168.60.20, pero el comando intentaba conectarse a local	Se forzó MYSQL_HOST=192.168.60.20 y --protocol=tcp en el script

Access denied con usuario incorrecto	MariaDB indicaba acceso denegado para dasc@192.168.60.30 sin contraseña	mysqldump no estaba leyendo el ~/.my.cnf correcto o no existía usuario dedicado	Se creó dasc_backup en MariaDB, se guardó /home/dasc/.my.cnf con permisos 600 y se validó la conexión
Ruta de config.env movida	El servicio systemd dejó de arrancar tras mover carpetas del proyecto	Se cambió el directorio del proyecto sin actualizar la ruta en el servicio	Se editó dasc-api.service para apuntar al nuevo path real del config.env

Estas incidencias son especialmente útiles ya que demuestran que el sistema tuvo una evolución y se perfeccionó en un entorno real con varias máquinas y distintos tipos de error: servicio, permisos, conectividad, credenciales y herramientas faltantes y se fue corrigiendo

12. Estado final del MVP y límites asumidos

El estado final del módulo de backups en el MVP es funcional y demostrable. El usuario puede abrir el panel web, acceder al apartado de copias, rellenar los campos, lanzar la operación y obtener una copia real de la base de datos MariaDB en otra máquina. También existe soporte para compresión (No disponible para Docker, solo máquinas reales) y para eliminación automática de copias antiguas mediante retención.

Elemento	Estado en el MVP	Observación
Backup completo	Implementado y real	Se ejecuta mediante mysqldump remoto contra la base de datos de 192.168.60.20
Interfaz de backups	Implementada	Tres modales con formulario, mismo estilo visual que el resto del panel
Feedback de resultado	Implementado	Banner visual mediante /backups?ok=1 u ok=0

Compresión GZIP	Implementada	Se activa según parámetro del formulario
Retención	Implementada	Basada en antigüedad por días con find -mtime
Historial real	No implementado en esta fase	La tabla del MVP es estática y sirve como maqueta funcional
Backup incremental real	Pospuesto a versión final	La interfaz existe, pero el motor del MVP sigue siendo un dump completo
Backup diferencial real	Pospuesto a versión final	Queda unido al trabajo con binlogs y metadatos de restauración
Restauración, descarga y borrado desde la UI	Pospuesto a versión final	Se planifican para la siguiente fase del producto

13. Mejoras previstas para la versión final

Una vez estabilizado el MVP, el siguiente paso natural es convertir las opciones incremental y diferencial en funcionalidad real. Durante las pruebas ya se investigó la activación de binlogs en MariaDB, pero se decidió no forzar esa complejidad dentro de la entrega del MVP para no comprometer estabilidad ni calendario.

- Implementar copias incrementales reales apoyadas en binary logs de MariaDB
- Implementar copias diferenciales reales basadas en la última copia completa
- Registrar metadatos de cada copia para poder reconstruir cadenas de restauración
- Generar un historial real en la tabla del panel leyendo el contenido del directorio de backups
- Añadir descarga y borrado desde la interfaz web
- Añadir restauración guiada con confirmación visual y validación previa
- Separar mejor la lógica de depuración y producción en la API, dejando endpoints de diagnóstico internos y no públicos
- Valorar subida opcional a almacenamiento externo si el proyecto final así lo requiere

14 Comandos representativos del montaje

Resumen operativo de comandos

```
# Panel-API
sudo systemctl restart dasc-api
sudo systemctl status dasc-api --no-pager
curl -i -X POST http://127.0.0.1:8000/backups/run …

# Backups
ssh dasc@192.168.60.30 "ls -lah /home/dasc/backups"
ssh dasc@192.168.60.30 "mysqldump --protocol=tcp --databases employees | head"
ssh dasc@192.168.60.30 "/usr/local/bin/backups_api.sh full employees /home/dasc/backups
test.sql gzip 30 '' ""

# BaseDeDatos
sudo mariadb
SHOW DATABASES;
SHOW BINARY LOGS;
SELECT User, Host FROM mysql.user WHERE User='dasc_backup';
```

Documentación de Logs

1. Objetivo del módulo de logs

El objetivo era disponer de una parte del proyecto capaz de mostrar información relacionada con accesos, eventos y métricas de sistema de una forma visual y defendible en el MVP. Para ello se decidió separar dos funciones: Cacti como herramienta de monitorización y Grafana como panel moderno de gráficas

2. Decisión técnica adoptada

- **Cacti** se utiliza para la parte de monitorización base y comprobación del entorno.
- **Grafana** se instala como plataforma de gráficas y dashboards. (aún no implementado)
- Se buscó que todo quedara automatizado mediante instaladores Bash, evitando depender de pasos manuales
- La instalación se realiza sobre máquinas reales/VM, no en Docker

3. Trabajo realizado con Cacti

Se desarrolló y depuró un instalador para dejar Cacti operativo en Ubuntu con Apache, PHP y MariaDB
Durante el proceso se corrigieron varios problemas reales hasta conseguir que el asistente web de Cacti pudiera continuar y que el panel quedará accesible correctamente

Bloque	Función realizada
Apache + PHP	Activación de mod_php y uso de mpm_prefork para que los .php no se des
PHP	Ajuste de memory_limit, max_execution_time y date.timezone para pasar l
MariaDB	Creación de la BD cacti y del usuario cactiuser con sus permisos.
Cacti	Importación del esquema SQL si faltaban tablas y generación correcta de c
Poller	Creación del cron para poller.php cada 5 minutos.
Permisos	Corrección de rutas y permisos para que el wizard pudiera avanzar.

4. Problemas reales y solución aplicada

Descarga de archivos PHP en el navegador. Se resolvió instalando libapache2-mod-php y

cambiando Apache a mpm_prefork
Error de sed al modificar php.ini. Se debía a caracteres especiales en la timezone. Se corrigió escapando correctamente el valor
Fallo de conexión de Cacti a la base de datos. Se detectó que el config.php no quedaba bien parcheado. Se sustituyó por escritura directa del archivo
Checks de MariaDB en rojo. Se añadió configuración específica para buffers, collation y parámetros recomendados por Cacti.
Problemas de permisos del wizard. Se ajustaron rutas y permisos para directorios de recursos y archivos necesarios como csrf-secret.php.

5. Estado final de Cacti

Al finalizar el proceso, Cacti quedó accesible desde el navegador, con el asistente de instalación funcional y con la base de datos, PHP, Apache y MariaDB correctamente integrados. Esto permitió cerrar la parte de monitorización base del módulo de logs.

6. Trabajo realizado con Grafana

Después de estabilizar Cacti, se planteó el siguiente paso del MVP: usar Grafana para las gráficas. La decisión técnica fue instalar Grafana por separado (aún no implementada) y prepararlo para trabajar con una base de datos propia de eventos de acceso (`dasc_logs`), manteniendo Cacti como monitorización

- Cambio de contraseña inicial de acceso.
- Diseño de un instalador separado del de Cacti para no mezclar responsabilidades.
- Preparación de una base de datos mínima (`dasc_logs`) y de una tabla `access_events` para futuros eventos de acceso.
- Definición de un datasource MySQL en Grafana como siguiente paso lógico del sistema.

7. Estructura resultante del módulo de logs

La estructura funcional que se decidió para el MVP queda resumida así:

Elemento	Función en el proyecto
Cacti	Monitorización base, comprobación del entorno y validación técnica.
Grafana	Visualización de gráficas y dashboards orientados a presentación.
MariaDB	Base de datos tanto para Cacti como para futuros eventos del módulo de
Frontend DASC	Punto de acceso con botones hacia Cacti y Grafana.

Evolución del proyecto y visión de la versión final

El desarrollo de **DASC Server Manager** se ha planteado desde el inicio como un proyecto evolutivo.
El objetivo del MVP desarrollado en esta entrega ha sido demostrar que es posible construir un sistema funcional capaz de **gestionar servicios, ejecutar copias de seguridad y monitorizar la actividad del sistema desde un panel web centralizado**.

Sin embargo, esta primera versión representa únicamente una **base funcional del sistema**, sobre la que se pretende seguir desarrollando nuevas capacidades hasta alcanzar una versión más completa orientada a entornos reales de administración de servidores.

En este apartado se describen las principales mejoras y ampliaciones que se plantean para la evolución del proyecto.

1. Sistema completo de restauración de backups

En el MVP actual el sistema permite crear copias de seguridad desde la interfaz web, pero el proceso de restauración todavía no se encuentra implementado de forma completa.

La versión final del sistema incorporará:

- restauración directa de backups desde el panel
- selección del punto de restauración
- confirmación previa de la operación
- registro automático del evento en el sistema de logs

De esta forma el administrador podrá **recuperar una base de datos completa desde el panel sin necesidad de ejecutar comandos manuales en el servidor**.

2. Programación automática de copias de seguridad

Actualmente las copias de seguridad se ejecutan manualmente desde la interfaz web.

En futuras versiones se pretende añadir un sistema de planificación automática de tareas, que permita:

- programar backups diarios, semanales o mensuales
- configurar políticas de retención automática
- ejecutar copias en horarios de baja carga del sistema

Esto permitirá automatizar completamente la gestión de copias de seguridad.

3. Implementación completa de backups incrementales y diferenciales

Aunque la interfaz del panel ya contempla los tres tipos de backup (completo, incremental y diferencial), en el MVP se ha priorizado la implementación de copias completas para simplificar el sistema.

La versión final del proyecto pretende incorporar:

- copias incrementales reales basadas en los cambios desde el último backup
- copias diferenciales basadas en el último backup completo
- optimización del almacenamiento mediante reducción de datos duplicados

Esto permitirá mejorar la eficiencia del sistema y reducir el espacio necesario para almacenar las copias.

4. Sistema de autenticación y control de acceso

El MVP está diseñado para ejecutarse en un entorno controlado, por lo que actualmente el panel no incorpora un sistema completo de autenticación de usuarios.

Para mejorar la seguridad del sistema se propone añadir:

- sistema de login para acceder al panel
- gestión de usuarios y roles
- control de permisos según tipo de usuario
- registro de accesos al sistema

Esto permitirá garantizar que solo usuarios autorizados puedan realizar operaciones críticas como restaurar bases de datos o reiniciar servicios.

5. Sistema de alertas y notificaciones

Otra mejora prevista es la implementación de un sistema de notificaciones automáticas que informe al administrador cuando se produzcan incidencias.

Entre las funcionalidades previstas se incluyen:

- alertas cuando falle una copia de seguridad
- avisos cuando un servicio se detenga inesperadamente
- notificaciones ante errores críticos del sistema

Estas alertas podrían enviarse mediante correo electrónico o integraciones con otros sistemas de monitorización

6. Ampliación del sistema de monitorización

El proyecto ya integra herramientas de monitorización para visualizar actividad del sistema, pero se podrían ampliar las capacidades añadiendo:

- monitorización del uso de CPU, memoria y disco
- gráficos históricos de rendimiento
- visualización de métricas directamente desde el panel

Estas mejoras permitirían convertir el sistema en una herramienta más completa de administración de servidores.

7. Mejora de la interfaz web

El panel actual se ha diseñado con el objetivo de ser funcional y fácil de utilizar. No obstante, se plantean mejoras en la experiencia de usuario.

Entre ellas:

- dashboard con estado general del sistema
- indicadores visuales del estado de servicios
- estadísticas de backups realizados
- mejora de la navegación entre módulos

Esto permitiría ofrecer una experiencia más clara e intuitiva para los administradores del sistema

8. Instalación y despliegue automatizado

Actualmente el sistema requiere varios pasos manuales para su instalación.

En futuras versiones se pretende mejorar este proceso mediante:

- scripts de instalación automática

- despliegue completo mediante Docker Compose
- configuración inicial automática del sistema

Esto permitiría instalar el sistema en nuevos servidores de forma rápida y reproducible.

Gestión de Tareas

Fase	Tasca	Estat	Responsable	Data Inici	Data Final
Fase 1	Creación de la idea del proyecto	Finalitzada	Tots	17-10-2025	26-10-2025
Fase 1	Proposición de ideas	Finalitzada	Tots	22-10-2025	26-10-2025
Fase 1	Análisis DAFO	Finalitzada	Tots	26 - 10 - 2025	26 - 10 - 2025
Fase 1	Análisis MOSCOW	Finalitzada	Tots	3 - 11 - 2025	3 - 11 - 2025
Fase 1	Identificación del problema	Finalitzada	Tots	7 - 11 - 2025	7 - 11 - 2025
Fase 1	Análisis 5 WHYS	Finalitzada	Tots	7 - 11 - 2025	7 - 11 - 2025
Fase 1	Definición departamentos empresa	Finalitzada	Tots	10 - 11 - 2025	10 - 11 - 2025
Fase 1	Plan de actividades inicial	Finalitzada	Tots	10 - 11 - 2025	15 - 11 - 2025
Fase 2	Diseño arquitectura del sistema	Finalitzada	Tots	17 - 11 - 2025	20 - 11 - 2025
Fase 2	Diseño red lógica Packet Tracer	Finalitzada	Daniel, Santino	23 - 11 - 2025	3 - 3 - 2026
Fase 2	Diseño FossFlow	Finalitzada	Santino	4 - 3 - 2026	6 - 3 - 2026
Fase 2	Diseño red física	Finalitzada	Daniel	10 - 12 - 2025	12 - 12 - 2025
Fase 2	Planificación de servidores	Finalitzada	Santino	9 - 1 - 2026	12 - 1 - 2026
Fase 3	Instalación Ubuntu Server	Finalitzada	Santino	13 - 1 - 2026	14 - 1 - 2026
Fase 3	Configuración red servidores	Finalitzada	Santino	15 - 1 - 2026	17 - 1 - 2026
Fase 3	Configuración SSH	Finalitzada	Aiman	17 - 1 - 2026	18 - 1 - 2026
Fase 3	Instalación MySQL	Finalitzada	Daniel	17 - 1 - 2026	19 - 1 - 2026
Fase 3	Configuración base de datos	Finalitzada	Daniel	19 - 1 - 2026	28 - 1 - 2026

Fase 3	Instalación Cacti	Finalitzada	Santino	30 - 1 - 2026	4 - 3 - 2026
Fase 3	Instalación Gráfana	En espera		30 - 1 - 2026	
Fase 3	Configuración monitorización	Finalitzada	Santino	2 - 2 - 2026	4 - 3 - 2026
Fase 3	Pruebas conexión entre servidores	Finalitzada	Santino	15 - 1 - 2026	10 - 2 - 2026
Fase 4	Creación API FastAPI	Finalitzada	Aiman	10 - 2 - 2026	20 - 2 - 2026
Fase 4	Configuración rutas API	Finalitzada	Aiman	20 - 2 - 2026	25 - 2 - 2026
Fase 4	Desarrollo backend sistema backups	Finalitzada	Santino	10 - 2 - 2026	25 - 2 - 2026
Fase 4	Integración ejecución scripts por SSH	Finalitzada	Aiman	10 - 2 - 2026	25 - 2 - 2026
Fase 4	Diseño estructura frontend	Finalitzada	Santino	26 - 1 - 2026	6 - 2 - 2026
Fase 4	Desarrollo panel web HTML	Finalitzada	Santino	26 - 1 - 2026	18 - 2 - 2026
Fase 4	Diseño estilos CSS	Finalitzada	Santino, Carlos	26 - 1 - 2026	18 - 1 - 2026
Fase 4	Integración frontend con API	Finalitzada	Aiman	25 - 2 - 2026	27 - 2 - 2026
Fase 5	Creación script backups	Finalitzada	Santino, Carlos	9 - 1 - 2026	15 - 2 - 2026
Fase 5	Integración mysqldump	Finalitzada	Daniel	9 - 1 - 2026	20 - 2 - 2026
Fase 5	Integración de copias Incrementales y Diferenciales	En espera		20 - 2 - 2026	
Fase 5	Sistema compresión backups	Finalitzada	Santino	20 - 2 - 2026	25 - 2 - 2026
Fase 5	Sistema retención backups	Finalitzada	Carlos, Santino	20 - 2 - 2026	25 - 2 - 2026
Fase 5	Ejecución backups desde API	Finalitzada	Santino	20 - 2 - 2026	25 - 2 - 2026
Fase 5	Integración backups en panel web	Finalitzada	Santino	20 - 2 - 2026	25 - 2 - 2026
Fase 6	Instalación sistema logs	Finalitzada	Santino	20 - 1 - 2026	4 - 3 - 2026
Fase 6	Configuración logs MySQL	Finalitzada	Santino	20 - 1 - 2026	4 - 3 - 2026
Fase 6	Integración logs en panel web	Finalitzada	Santino	20 - 1 - 2026	6 - 3 - 2026

Fase 7	Creación script control servicios	Finalitzada	Daniel, Aiman	20 - 1 - 2026	27 - 2 - 2026
Fase 7	Integración control servicios API	Finalitzada	Aiman	25 - 1 - 2026	27 - 2 - 2026
Fase 7	Creación panel gestión servicios	Finalitzada	Aiman	25 - 1 - 2026	27 - 2 - 2026
Fase 8	Integración de Cacti en Docker	Finalitzada	Santino	27 - 2 - 2026	4 - 3 - 2026
Fase 8	Creación Docker Compose	Finalitzada	Santino	27 - 2 - 2026	4 - 3 - 2026
Fase 8	Configuración contenedores	Finalitzada	Santino	27 - 2 - 2026	4 - 3 - 2026
Fase 8	Pruebas contenedores Docker	Finalitzada	Santino	27 - 2 - 2026	4 - 3 - 2026
Fase 9	Plan de Sostenibilidad del Proyecto	Finalitzada	Carlos	20 - 2 - 2026	4 - 3 - 2026
Fase 9	Documentación técnica API	Finalitzada	Aiman	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Documentación Packet Tracer	Finalitzada	Daniel	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Documentación Frontend (HTML/CSS)	En espera	Santino	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Documentación Servicios	Finalitzada	Aiman	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Documentación Docker	Finalitzada	Santino	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Documentación Logs y Cacti	En curs	Santino	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Estructura Documentación	Finalitzada	Carlos	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Creación diagramas arquitectura	Finalitzada	Carlos, Daniel	2 - 3 - 2026	6 - 3 - 2026
Fase 9	Preparación presentación MVP	En curs	Tots	6 - 3 - 2026	